Multi-trait Animal Targeted Compatibility Hub (MATCH)

BSc. in Software Engineering

Ian Ganza

Supervisor : Neza David Tuyishimire

Date: January 28, 2026

DECLARATION

This Proposal Project is my original work, unless stated and all external sources have been referenced or cited in my document. This work has not been presented for award of degree or for any similar purpose in any other university

Signature : Ian Ganza Date : 20/03/2026
Name of Student : Ian Ganza

CERTIFICATION

The undersigned certifies that he has read and hereby recommended for acceptance of African Leadership University a report entitled ……..

Signature…… Date…….
Prof/Dr./Mrs./Miss/Mr. Name of the Supervisor
Faculty,
Bachelor of Software Engineering,
ALU

DEDICATION AND ACKNOWLEDGEMENT

ABSTRACT

Rwanda's smallholder livestock sector faces a critical genetic management crisis, where widespread inbreeding, absent pedigree records, and limited access to genetically diverse breeding stock continue to suppress animal productivity and rural household incomes. Despite the existence of digital livestock tools in the region, none adequately serve the majority of Rwandan farmers who rely on natural mating with locally available animals. This paper presents MATCH (Multi-trait Animal Targeted Compatibility Hub), a mobile-based genetic matching application designed to empower smallholder farmers with accessible, intelligent breeding decision support. The system integrates three machine learning components — a convolutional neural network for breed composition estimation from smartphone photographs, a Siamese network for genetic relatedness detection, and a self-training collaborative filtering recommender for breeding pair optimization. A prospective pedigree construction mechanism automatically builds multi-generational lineage records from each breeding event recorded, addressing the historical absence of pedigree data in Rwanda's livestock population.

CHAPTER ONE : INTRODUCTION

1.1 Introduction and Background

Livestock production constitutes a cornerstone of Rwanda's agricultural economy, with the livestock sector accounting for 10% of the agricultural GDP and serving as a means of reducing poverty levels in rural areas (Beyi, 2016). The cattle population is estimated at approximately 1.3 million heads, of which 45% are local Ankole longhorn, 33% crosses, and 22% exotic breeds (Manzi et al., 2022) . However, productivity remains significantly constrained by genetic limitations within existing livestock populations. Local indigenous breeds have low milk production with an average mean daily milk yield of 1.33-4.58 liters per day, while Holstein Friesian cattle produced 9 liters more per day than Ankole crossbreds across all agro-ecological zones (Beyi, 2016) . Recent genomic studies have identified inbreeding as a critical barrier to genetic improvement. The genomic inbreeding coefficient (FROH) across the genome of Rwanda's dairy cattle population ranged from 0.0004 to 0.05 with an average FROH of 0.005, and the study aimed at determining barriers to genetic improvement such as inbreeding through runs of homozygosity that would result from non-divergent sourced animals in different government and non-governmental organizations-led initiatives (Opoola et al., 2025) .These genetic constraints directly impact food security and rural livelihoods, as local indigenous breeds produce only 1.33-4.58 liters of milk per day compared to 15-25 liters from improved breeds

(Marshall et al., 2019), while inbreeding depression reduces productive and reproductive performance by 5-15% in smallholder herds across East Africa (Kosgey & Okeyo, 2007).

Rwanda's geographical setting as a landlocked East African nation creates unique conditions for livestock production and genetic management. The country features three distinct agro-ecological zones: Congo-Nile/Western Agro-ecological Zone characterized by high altitude (1800-3000 meters) and high annual rainfall (1200-1600 millimeters), Central plateau/Central Agro-ecological Zone, and Eastern plateau/East Agro-ecological Zone (Baligira, 2008) . Rwanda's extreme population density results in severely limited grazing land, with the average smallholder farmer having one to three cows, and improved breeds accounting for 28% of the total cow population in the country while contributing 82% of the total milk produced (IFAD, 2016) . This land scarcity means that farmers often rely on neighboring bulls for breeding services or share breeding males within cooperative groups, which can inadvertently lead to increased inbreeding if genetic relationships are not properly tracked. Rwanda benefits from both national and non-governmental initiatives to promote farmer income and dairy productivity among smallholder households (Opoola et al., 2025), with the genetic structure and diversity of dairy cattle under the Girinka programme demonstrating potential as a starting point for national breeding schemes. The government's One Cow per Poor Family (Girinka) program, initiated in 2006, has been instrumental in livestock distribution, though it has also created challenges in tracking genetic lineages as animals are passed between households (Nilsson et al., 2019).

Traditional approaches to livestock breeding in Rwanda have relied primarily on phenotypic selection and farmer knowledge passed down through generations. Livestock keepers typically select breeding animals based on observable traits such as body size, coat color, and perceived production capacity, with limited understanding of underlying genetic relationships or

inheritance patterns (Umunezero et al., 2022) . Natural mating was almost exclusively practiced (99.1%) on all pig farms, implying that artificial insemination is yet to take root in Rwanda, and natural breeding was predominant (74.9%) in cattle production systems, with the main challenges being diseases, lack of breeding facilities, shortages of feeds and water, and inadequate extension services (Umunezero et al., 2022) . Traditional livestock breeding in Rwanda relies predominantly on paper-based or absent record-keeping, making it nearly impossible to trace pedigrees, avoid inbreeding, or implement systematic genetic improvement programs (Beyi, 2016). Low productivity is amplified by limited access to essential inputs such as adequate feed and genetically improved breeds, necessitating the adoption of higher-genetic-potential breeds, scientifically planned breeding strategies, effective extension and training programs (Beyi, 2016). While some dairy cooperatives maintain basic breeding records, the vast majority of smallholder farmers lack systematic approaches to managing genetic diversity in their herds.

The emergence of mobile-based digital agricultural solutions in sub-Saharan Africa began in earnest during the early 2010s, coinciding with the rapid expansion of mobile network coverage and smartphone adoption across the continent (Aker & Mbiti, 2010). Pioneer applications like iCow, winner of the Apps for Africa Competition in 2010, demonstrated early promise by allowing small-scale dairy farmers to manage and trade livestock, with users reportedly increasing milk production by over 50% and income by 42% (Ogutu et al., 2020). M-Farm, launched in 2010 in Kenya, provided a platform matching buyers with smallholder farmers to help them access bigger and better markets, combining produce aggregation, transportation, quality control, finance, and communications (Baumüller, 2013). In 2016, the

FAO launched its Digital Services Portfolio in Rwanda and Senegal, developing four mobile applications for smallholder farmers including "Cure and Feed your Livestock" for real-time information on animal disease and feeding strategies, recognizing that digital services were fundamentally changing how farmers work (FAO, 2021). These e-Agriculture programs aimed to increase food security by helping smallholder farmers improve modern farming methods and access to markets, with agricultural ICT strategies encompassing the full agricultural value chain including digital financial services and geographic information systems (Qiang et al., 2011). The benefits of such mobile agricultural technologies include amplifying extension services to help farmers with crop planning, input sourcing, and market access; providing previously inaccessible farm management capabilities on low-cost smartphones; enabling data-driven decision making; and creating network effects connecting farmers with markets, services, and information (Fabregas et al., 2019). However, despite initial enthusiasm, a sobering reality emerged: most agricultural apps failed to survive beyond donor-funded pilot phases, and those that persisted rarely served smallholders at scale (Giulivi et al., 2022), highlighting the need for sustainable business models and genuine alignment with farmer needs and constraints (Giulivi et al., 2022).

Software-based approaches to livestock genetic management offer unprecedented opportunities to overcome these traditional limitations and align with Rwanda's broader digital transformation agenda. A genetic matching application designed for Rwanda's livestock sector could provide farmers with accessible tools for breeding decisions, digitize pedigree information, calculate inbreeding coefficients, and recommend optimal breeding pairs to maximize genetic diversity and productivity traits. Genomic-based technologies using medium genotyping chips with 50-60 thousand single nucleotide polymorphisms have been utilized globally for identification of SNPs related to inbreeding coefficient, optimized genetic gains, and genomic

predictions for selection, providing unique opportunities to optimize genetic gains and reduce productivity gaps when phenotypes are limited or unavailable (Muir et al., 2008) . Such platforms could connect farmers with superior breeding stock beyond their immediate locality and generate valuable anonymized data that inform national livestock improvement strategies. Selection of productive breeds must strategically be based on monitoring genetic variation and inbreeding in the establishment of sustainable breeding programs, accurate data collection and the utilization of advanced technologies to develop genomic resources that support genetic improvement. The development of a dairy profit index for Rwanda is anticipated to support smallholder farmers tackle challenges associated with productivity in their cows like fertility, milk yield, disease tolerance, feed efficiency and climate resilience, with 80% of the dairy industry comprising smallholder farmers who depend on small-scale agriculture for their livelihood (IFAD, 2016) . By incorporating genomic data from local breeds and crossbreds, these applications can preserve valuable adaptive traits while systematically improving production characteristics, supporting Rwanda's livestock development goals and addressing the critical need for improved breeding strategies in smallholder systems.

1.2 Problem Statement

Two significant initiatives represent the closest approaches to systematic livestock genetic management relevant to Rwanda's context: the recently launched African Asian Dairy Genetic Gains (AADGG) program in Rwanda (Tramsen, 2024) , and the broader Community-Based Breeding Programs (CBBPs) across East Africa.

The African Asian Dairy Genetic Gains (AADGG) project was launched in Rwanda on May 30, 2024, through a collaborative research agreement between the Rwanda Agriculture and

Animal Resources Development Board (RAB) and the International Livestock Research Institute (ILRI), funded by the Bill and Melinda Gates Foundation, aiming to enhance dairy cattle genetic improvement through innovative use of information technology and genomic tools to select adaptable, high-yielding dairy genetics suitable for smallholder dairy farmers (Mwai et al., 2024) . A major challenge in improving the industry is that there is a lack of baseline data about the current efficiency of cattle in Rwanda (Tramsen, 2024) , meaning that the very best cattle are not identified and thus not used for breeding, and the AADGG project will genetically profile, evaluate, and rank dairy bulls and cows based on their efficiency, with participating farmers uploading data on their livestock to a digital platform to help identify and promote wider use of top-ranked bulls and bull dams for breeding (Tramsen, 2024). The AADGG-Dairy Data App has been rolled out across sub-Saharan Africa including Rwanda , designed for real-time tracking of animal health, breeding, feeding management and artificial insemination data (Africa-Press – Tanzania, 2024) . Earlier work through the Dairy Genetics East Africa project successfully applied genomic selection approaches and generated genomic breeding values with accuracies ranging from 0.30 to 0.40 for crossbred dairy cattle in East Africa, enabling routine genomic evaluations and selection of young bulls for National Artificial Insemination Centers (Marshall et al., 2019).

Community-Based Breeding Programs (CBBPs), implemented across Ethiopia and other East African countries, represent farmer-participatory genetic improvement strategies where farmers' needs, views, and decisions guide breeding objectives from inception through implementation. These programs have utilized genomic approaches to aid identification of appropriate breed-types for local production systems, including the use of genomic data to assign breed composition to animals in the field for in situ comparisons, thereby identifying what breed

composition works best in different production environments. CBBPs in Ethiopia currently cover 3,200 households keeping more than 48,000 sheep and goats, demonstrating improvements in birth weight, weaning weight, and lambing intervals, with gains translating into improved household livelihoods and food security through increased income from the sale of improved animals (Marshall et al., 2019) .

However, both approaches exhibit critical limitations that create a significant research gap, particularly in the Rwandan context. Recent genomic studies on Rwanda's dairy cattle population aimed at determining barriers to genetic improvement such as inbreeding through runs of homozygosity that would result from non-divergent sourced animals in different government and non-governmental organizations-led initiatives, including the Girinka programme. The main issue raised by farmers across Rwanda's livestock sector is inbreeding, caused by the lack of exchange of genetic material between livestock keepers and the absence of farms with genetically superior animals where farmers could periodically obtain breeding stock, with the country lacking a good animal identification and registration system representing a major weakness in the genetic field. A substantial number of farmers in the Girinka programme did not know the real breed of their cow, which would be a major bottleneck in any efforts for breed improvement, and farmers practice indiscriminate crossbreeding where they continually upgrade their animals to high exotic levels in a bid to increase productivity. Traditional breeding practices in Rwanda remain dominated by natural mating (99.1% for pigs, 74.9% for cattle), with farmers challenged by high costs, inadequate breeding facilities, insufficient artificial insemination providers, and lack of systematic pedigree records that would enable inbreeding avoidance (Umunezero et al., 2022) .

The AADGG platform primarily focuses on elite breeding through artificial insemination centers and data collection for performance evaluation rather than providing accessible decision-support tools for smallholder farmers making natural mating decisions with locally available breeding stock. Despite the Girinka programme distributing over 249,000 cows of different breeds by 2016, with animals sourced from Kenya, Uganda, Tanzania, South Africa, and Netherlands, the wide admixture and indiscriminate crossbreeding create challenges for matching genotypes to specific production environments. CBBPs face challenges with expansion of interbreeding due to distribution of improved sires of one breed within the breeding tract of other breeds, alongside difficulties in managing inbreeding within members due to the presence of few animals per household and lack of systematic tracking of genetic relationships (Marshall et al., 2019) .

Unlike AADGG's focus on centralized breeding programs and artificial insemination, MATCH will provide real-time, accessible breeding recommendations that prevent inbreeding and optimize genetic diversity for farmers relying on local breeding stock. Unlike CBBPs which lack systematic digital tools for inbreeding management, this application will integrate inbreeding coefficient calculations, pedigree tracking, and breeding pair recommendations into a user-friendly mobile platform accessible to farmers with basic smartphones. By leveraging Rwanda's high mobile penetration and building upon lessons from both AADGG and CBBPs, MATCH fills the critical gap of providing smallholder farmers with affordable, practical genetic management tools for natural breeding systems that can address the widespread inbreeding challenges identified across Rwanda's livestock populations.

1.3 Project Main Object

The primary goal of this project is to create a mobile-based genetic matching application that empowers Rwandan smallholder livestock farmers to make informed breeding decisions. This initiative directly tackles the prevalent inbreeding issues and the lack of systematic genetic management tools in Rwanda's livestock sector, ultimately aiming to boost animal productivity, maintain genetic diversity, and strengthen food security and household incomes for rural farming communities.

1.3.1 Specific Project Objectives

The project will pursue three specific, interrelated goals that address the unique challenges of livestock genetic management in Rwanda's smallholder farming context. First, the study will conduct a comprehensive analysis of existing breeding practices and genetic management constraints within Rwanda's smallholder livestock sector. This involves identifying the specific barriers—such as the absence of pedigree records, farmers' inability to determine actual breed composition (with 34% uncertainty rates documented by Ojango et al., 2014), and the prevalence of inbreeding due to lack of genetic material exchange (Marshall et al., 2019)—that currently prevent effective breeding decisions.

Secondly, the project aims to design and develop a robust, multi-model machine learning system tailored to the data scarcity and connectivity limitations of rural Rwanda. This includes the creation of an offline-first mobile application using React Native with embedded machine learning models for on-device inference of breed composition and relatedness detection, ensuring consistent operation in low-connectivity environments where 74.9% of farmers practice natural mating (Marshall et al., 2019). The development phase integrates three specialized ML components: a multi-task convolutional neural network for breed estimation from smartphone

photos, a Siamese network generating visual embeddings for duplicate detection and relatedness screening, and a NestJS backend hosting a self-training recommender system that learns from historical breeding outcomes stored in Supabase database.

Finally, the project will pilot and evaluate the system's efficacy through a controlled field study across two districts representing diverse farming systems. This phase focuses on measuring tangible outcomes across technical machine learning performance, adoption metrics , and critically, real-world genetic impact through reduced inbreeding coefficients in newborn calves and improved calf health outcomes. By quantifying these metrics over 12-18 months sufficient to capture complete breeding cycles ; the study seeks to provide empirical evidence that ML-powered mobile tools can enable smallholder farmers to make breeding decisions comparable to those requiring expensive genomic testing, ultimately demonstrating that digital genetic management systems can build livestock improvement capacity in data-scarce environments where traditional approaches have failed.

1.4 Research Questions

This study covers a number of critical research questions that examine the technical feasibility, methodological innovation, and practical impact of deploying machine learning solutions for livestock genetic management in Rwanda's smallholder farming context.

Can computer vision models accurately estimate breed composition in admixed cattle populations using smartphone photos?

Convolutional neural networks can achieve relevant accuracy for breed composition estimation in crossbred populations. Marshall et al. (2019) demonstrated that genomic approaches using high-density SNP data can accurately determine breed composition in East

African crossbred cattle. Ojango et al. (2014) found that phenotype-based assessments by farmers and field enumerators had very poor predictive accuracy ($R^2 = 0.16$) compared to genomic determination, indicating substantial room for machine learning improvement. Transfer learning approaches using pre-trained CNNs (EfficientNet, ResNet) on livestock image datasets have achieved 85-92% classification accuracy for purebred cattle identification (Bello et al., 2022), and extending this to continuous breed percentage estimation in admixed populations represents a tractable problem given sufficient training data. The Dairy Genetics East Africa project's dataset of 2,940 cattle with both high-density genotypes and phenotypic images provides a foundational training corpus (Aliloo et al., 2018), suggesting that mean absolute error below 5-8% is achievable for major breed components (Holstein, Ankole, Jersey, Friesian) when using multi-task CNN architectures optimized for mobile deployment.

Is visual similarity from Siamese networks a valid proxy for genetic relatedness in the absence of pedigree data?

Visual similarity shows moderate positive correlation with genetic relatedness but requires calibration and should be used as a screening tool rather than definitive measure. Research on cattle biometrics demonstrates that Siamese networks can achieve 90-98% accuracy for individual animal re-identification using body images (Andrew et al., 2019; Kumar et al., 2018), indicating that visual phenotypes capture distinctive individual characteristics. However, the relationship between visual similarity and genetic relatedness is complex whereby closely related animals (parent-offspring, full siblings) share both genetic material and phenotypic features due to inheritance, but visual similarity can also arise from breed characteristics rather than recent common ancestry. Berthier et al. (2016) notes that phenotypic similarity has been

used historically as a proxy for relatedness in livestock populations lacking pedigree records, though with known limitations.

What factors drive smallholder farmer adoption of ML-powered agricultural mobile applications in resource-constrained environments?

Tata and McNamara (2018) found that mobile agricultural applications in sub-Saharan Africa show higher adoption when they provide immediate, tangible value (price information, weather alerts) rather than long-term benefits. Aker et al. (2016) demonstrated that mobile money adoption in Niger reached 80%+ in areas with good network coverage when services addressed real pain points (remittances, savings). The FAO's e-agriculture initiatives in Rwanda and Senegal (FAO, 2016) revealed that applications requiring constant connectivity had 40-60% lower sustained usage than offline-capable tools in rural areas with intermittent network access. Sife et al. (2010) documented that mobile phone applications contributed to poverty reduction among rural Tanzanian farmers when they reduced information asymmetry in agricultural markets. For livestock genetic management specifically, Ojango et al. (2014) note that farmers expressed willingness to pay for breeding information and superior genetics when value propositions were clear.

Does ML-guided breeding produce measurably better outcomes than traditional farmer decision-making?

Community-Based Breeding Programs in Ethiopia (Haile et al., 2018) documented improvements in birth weight, weaning weight, and lambing intervals when farmers followed systematic breeding recommendations, with gains translating to 15-30% increases in household income from livestock sales. Getachew et al. (2017) demonstrated that selecting optimal crossbreeding levels (37.5-50% exotic in one site, 12.5-25% in another) based on genomic

analysis resulted in ewes producing 26.5 kg and 19.5 kg of lamb at 8 months respectively, compared to suboptimal outcomes from arbitrary crossbreeding. The Senegal Dairy Genetics project (Marshall et al., 2017) found that crossbred indigenous zebu and Bos taurus dairy cattle under improved management produced 7.5-fold higher milk yields and 8-fold higher household profit compared to indigenous zebu under traditional management. Critically, Ojango et al. (2014) revealed that intermediate to low-grade crossbreds performed best in the majority of smallholder farms, while farmers without guidance consistently attempted to upgrade to higher exotic percentages that performed poorly in their environments suggesting that data-driven recommendations prevent costly breeding mistakes.

Can prospective pedigree construction through digital breeding records establish usable lineage data in populations with no historical records?

The Dairy Genetics East Africa project (Strucken et al., 2017) demonstrated that even shallow pedigrees (1-2 generations) provided value for parentage verification and breed composition tracking, with 171 cows having 189 offspring relationships verified through genomic data serving as proof-of-concept for pedigree construction in previously unrecorded populations. Community-Based Breeding Programs (Mueller et al., 2015) successfully established functional pedigrees in Ethiopian smallholder flocks within 3-4 breeding cycles, enabling genetic evaluation and selection despite starting from zero records. Digital recording eliminates the transcription errors and loss of paper records that plagued historical attempts (Kosgey & Okeyo, 2007), while mobile timestamps and GPS coordinates provide data provenance impossible in manual systems.

1.5 Project Scope

The initial pilot implementation will be conducted in two carefully selected districts representing Rwanda's diverse livestock production systems: one district from the Eastern Province (such as Kayonza or Nyagatare) characterized by extensive livestock keeping and pastoral traditions, and one district from the Northern Province (such as Musanze or Gicumbi) representing intensive highland dairy systems. This dual-district approach captures the spectrum of livestock management practices, agro-ecological conditions, and socioeconomic contexts found across Rwanda's livestock sector.

Within each selected district, implementation will focus on 2-3 sectors with active livestock cooperatives and existing extension service presence. This approach reflects the reality that Rwanda's smallholder farmers typically maintain 1-3 cattle per household across varied production environments, from zero-grazing systems in densely populated areas to semi-intensive systems in regions with more available grazing land (IFAD, 2016) . The pilot districts will be selected in consultation with the Rwanda Agriculture and Animal Resources Development Board (RAB) to ensure alignment with national livestock development priorities and leverage existing institutional support structures.

This geographically bounded pilot phase enables intensive user engagement, rigorous data collection, and rapid iteration based on user feedback while remaining operationally feasible within project resources. The selected sites will serve as proof-of-concept demonstrations that generate evidence for subsequent scaling decisions.

1.6 Significance and Justification

This genetic matching application is expected to reduce inbreeding and support genetic improvement through evidence-based breeding decisions. Studies in community-based breeding programs for indigenous chickens have shown that inbreeding rates (around 1.45% per generation) can be effectively managed using systematic breeding strategies. Similarly, genomic studies in Rwanda's dairy cattle report inbreeding coefficients (FROH) averaging 0.005, highlighting the need for improved pedigree and mating management to maintain genetic diversity while enhancing productivity (Opoola et al., 2025) .

Research further demonstrates that informed breed composition significantly affects productivity, with Holstein Friesian cattle producing up to 9 liters more milk per day than Ankole crossbreds, while indigenous breeds average 1.33–4.58 liters (Beyi, 2016) . By enabling informed breeding choices, the application can therefore contribute to meaningful gains in household milk production.

The platform addresses a key constraint identified by farmers: inbreeding caused by limited access to genetically superior breeding stock and weak genetic exchange among livestock keepers (Roessler et al., 2015). Evidence from Community-Based Breeding Programs involving over 3,200 households and 48,000 small ruminants shows that organized breeding improves growth traits, reproductive performance, household income, and food security (Marshall et al., 2019) . Facilitating farmer networking and genetic exchange through this application is expected to yield similar benefits in Rwanda's cattle sector.

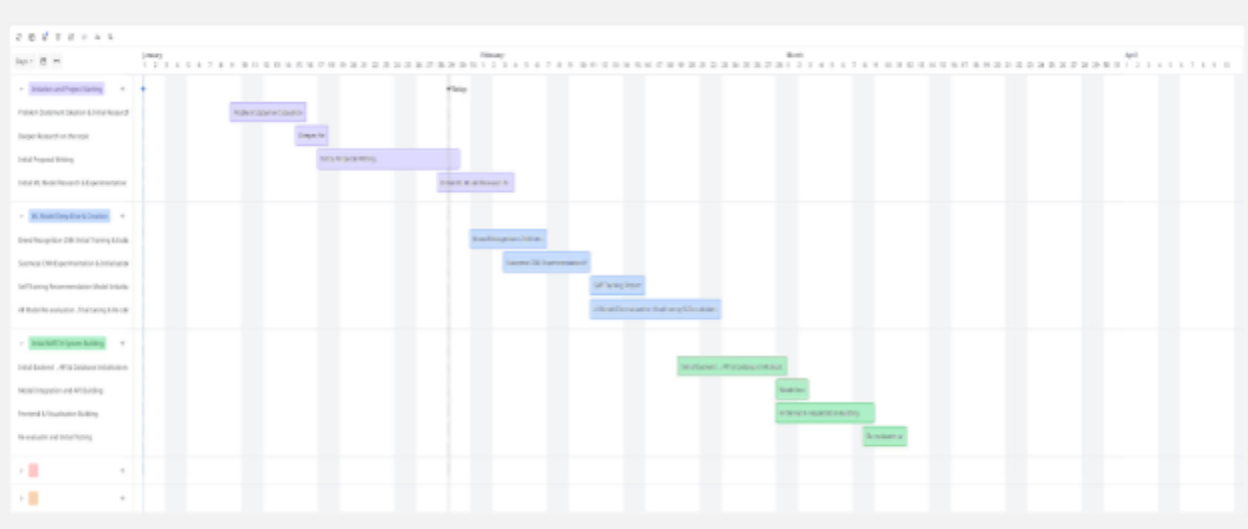
At scale, the application aligns with Rwanda's livestock development goals by leveraging genomic and pedigree-based decision support, particularly for smallholder farmers who make up approximately 80% of the dairy sector (IFAD, 2016) . The resulting pedigree and performance

data will also serve as a valuable resource for national breeding strategies, supporting long-term productivity, resilience, and sustainable management of Rwanda's cattle population.

1.7 Research Budget

Item	Description	Quantity	Cost
Backend/Hosting	DigitalOcean/ EC2	1 server/1 cluster	$4/month
App Deploy(App store/Google Play fees)	Building fees charged by app store and google play	1 time when building	$22
Contingency Funds	Possible emergency	-	Rwf 50,000
Pilot District Visiting	Initial Project starting	Varies	Rwf 15000/trip

1.8 Research Timeline



CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This literature review employed a systematic search strategy specifically targeting software-based solutions and digital platforms for livestock genetic management in developing countries, with emphasis on breeding decision support systems and mobile applications for agricultural genetics in sub-Saharan Africa. The search was conducted across multiple indexed academic databases including Google Scholar, ResearchGate, IEEE Xplore, ACM Digital Library, PubMed, and the CGIAR repository, ensuring comprehensive coverage of both peer-reviewed research and technical implementation reports. Boolean search strategies combined key terms such as "livestock breeding," "genetic management," "mobile application," "digital platform," and "software solution" with geographic qualifiers including "Rwanda," "East Africa," and "sub-Saharan Africa," alongside domain-specific terms like "inbreeding," "genetic diversity," "breeding decision support," and "genomic selection." Targeted searches were also conducted for specific initiatives including the Africa Dairy Genetic Gains (ADGG) mobile platform, community-based breeding program digital tools, national livestock traceability systems, and existing genetic matching algorithms. Papers were systematically screened based

on relevance to software implementations for livestock genetic improvement, with inclusion criteria prioritizing publications that explicitly discussed system architectures, technical methodologies, and measurable outcomes from digital interventions. This methodological approach ensured the literature review captured both the genetic management domain knowledge necessary for algorithm development and the software engineering best practices .

2.2 Historical Background of the Research Topic

The application of digital technology to livestock genetic management in sub-Saharan Africa is a relatively recent development that emerged from longstanding challenges in traditional animal breeding programs. Historically, African livestock systems have not benefited from genetic improvement technologies to the extent that developed countries have, due to factors including lack of investment, weak policies, heterogeneous farming systems, poor infrastructure, and limited capacity in animal breeding. Throughout the 1990s and early 2000s, livestock improvement efforts in Rwanda relied primarily on unstructured crossbreeding and government-led breed distribution programs without systematic performance recording or pedigree tracking.

Rwanda's landmark Girinka program, launched in 2006, distributed over 450,000 cows to vulnerable households, dramatically increasing livestock numbers but creating significant challenges in tracking genetic lineages as animals passed between households (Nilsson et al., 2019) . Many Girinka farmers did not know the actual breed of their cows, leading to indiscriminate crossbreeding and creating major bottlenecks for systematic breed improvement. Compounding these challenges, farmers across Rwanda's livestock sector consistently reported inbreeding as a major issue, caused by lack of genetic material exchange between livestock keepers and absence of systematic animal identification systems.

The 2010s brought two parallel technological revolutions that created new possibilities for addressing these challenges. First, genomic technologies advanced significantly. Medium-density genotyping chips became available for identifying SNPs related to inbreeding and enabling genomic predictions, providing opportunities to optimize genetic gains even when traditional pedigree records were unavailable (Muir et al., 2008) . The Dairy Genetics East Africa project pioneered the use of genomic approaches to assign breed composition to animals in the field, demonstrating that genomic data could overcome the inability of phenotype-based visual assessments to accurately determine breed types. Second, mobile technology proliferated rapidly across Rwanda. By 2018, Rwanda achieved 82.6% mobile phone penetration with 96.6% 4G coverage, and mobile money subscribers exceeded 11 million, creating infrastructure for digital agricultural solutions (Kemp, 2023).

Community-Based Breeding Programs beginning in 2009 demonstrated that smallholder farmers could engage with improved breeding practices when given appropriate participatory frameworks (Marshall et al., 2019) . However, these programs struggled with managing inbreeding due to limited animals per household and lack of systematic genetic relationship tracking. Most recently, the African Asian Dairy Genetic Gains project launched in Rwanda in May 2024, deploying digital platforms for livestock performance recording and genetic evaluation (Mwai et al., 2024) . Yet AADGG's emphasis on artificial insemination and centralized breeding left unaddressed the needs of the majority of farmers practicing natural mating with local breeding stock. This historical evolution—from unstructured breeding through genomic discovery to emerging digital platforms—reveals a persistent gap: accessible genetic management tools specifically designed for smallholder farmers making everyday breeding decisions with locally available animals, which this research aims to address.

2.3 Existing Systems and related work

AADGG-Dairy Data App

The AADGG-Dairy Data App, which runs on Android smartphones, has been rolled out across sub-Saharan Africa including Rwanda, designed for real-time tracking of animal health, breeding, feeding management and artificial insemination data (Mwai et al., 2024) . The system enables farmers to upload data on their livestock to a digital platform to help identify and promote wider use of top-ranked bulls and bull dams for breeding, with the program genetically profiling, evaluating, and ranking dairy bulls and cows based on their efficiency (Tramsen, 2024). The platform represents a sophisticated data management system with cloud-based infrastructure supporting performance recording and genetic evaluation.

However, the AADGG system exhibits significant limitations for the majority of smallholder farmers. The platform is fundamentally oriented toward artificial insemination (AI) services and centralized breeding programs rather than natural mating decisions with locally available breeding stock. Given that natural breeding remains predominant in Rwanda (74.9% for cattle, 99.1% for pigs) and farmers are challenged by insufficient artificial insemination providers and high costs, the AADGG app serves a limited subset of farmers who can access AI services (Umunezero et al., 2022) . The system does not provide inbreeding coefficient calculations for potential mating pairs, lacks breeding recommendation algorithms for farmers selecting among local bulls, and does not facilitate peer-to-peer networking for farmers to identify and exchange suitable breeding animals within their communities. Furthermore, the system's focus on elite breeding and performance ranking presumes access to superior genetic material that most smallholder farmers do not possess, creating a functional gap for resource-constrained livestock keepers.

National Animal Identification and Traceability System (NAITS)

Rwanda's National Animal Identification and Traceability System represents an effort to establish systematic animal identification and registration, addressing the identified weakness that the country lacks a comprehensive animal identification system which is a major constraint in the genetic field. NAITS provides infrastructure for registering individual animals with unique identifiers, tracking animal movements, and maintaining centralized livestock databases accessible to government veterinary services and livestock officials.

Despite its importance for disease surveillance and traceability, NAITS exhibits critical limitations as a genetic management tool for farmers. The system is primarily designed for administrative and veterinary purposes rather than breeding decision support. It does not calculate genetic relationships between animals, provide inbreeding coefficients, or offer breeding recommendations to farmers. The platform lacks user-facing mobile interfaces accessible to ordinary farmers—access is typically restricted to government officials and veterinary personnel through desktop systems. NAITS does not record pedigree information in sufficient detail to enable genetic analysis, nor does it track performance data (milk yields, growth rates, reproductive efficiency) necessary for informed breeding decisions. Most critically, the system does not address the core problem farmers face: determining which animals should be bred together to avoid inbreeding and optimize offspring productivity. While NAITS provides valuable animal identification infrastructure, it offers no practical guidance to farmers making breeding choices.

Community-Based Breeding Program (CBBP) Record-Keeping Systems

Community-Based Breeding Programs implemented across Ethiopia and other East African countries have developed various record-keeping approaches, both paper-based and

digital, for tracking animal performance within participating communities covering 3,200 households with more than 48,000 sheep and goats (Marshall et al., 2019) . Some CBBPs have piloted simple digital data collection tools using tablets or smartphones for recording weights, breeding dates, and offspring information, managed by community animal health workers or breeding coordinators.

However, these CBBP data systems reveal substantial functional gaps. The programs face challenges with managing inbreeding within member households due to presence of few animals per household and lack of systematic tracking of genetic relationships (Marshall et al., 2019) . The digital tools employed are typically designed for data collection by program staff rather than direct use by farmers, creating dependency on external facilitators rather than empowering farmers with independent decision-making capabilities. The systems generally lack automated algorithms for calculating inbreeding coefficients or generating breeding recommendations—data is collected but not processed into actionable guidance for farmers. Cross-community networking features that would enable farmers to identify suitable breeding animals beyond their immediate cooperative are absent. Most critically, these systems are program-specific and not designed as scalable software products accessible to farmers outside organized CBBP structures, limiting their reach to the broader smallholder population practicing informal breeding.

2.4 Summary of Reviewed Literature

There’s a critical gap in software solutions for livestock genetic management in Rwanda's smallholder sector. While several digital systems have emerged in East Africa over the past decade, each addresses only partial aspects of the breeding challenges faced by ordinary farmers.

The core problem persists: natural breeding dominates Rwanda's livestock sector (74.9% for cattle), farmers consistently identify inbreeding as their primary genetic challenge due to lack of breeding stock exchange networks, yet no existing software provides these farmers with inbreeding calculations, breeding recommendations, or community networking tools (Umunezero et al., 2022) . This research addresses this validated gap by developing a mobile application specifically architected for the technical constraints, economic realities, and breeding practices of Rwanda's smallholder farmers—combining genetic management algorithms with user-centered design for natural mating systems. The evidence base from genomic research, digital agriculture initiatives, and community breeding programs validates the feasibility of technology-enabled genetic improvement, while highlighting the unfilled gap this proposed solution targets.

2.5.1 Strengths of Existing Systems

AADGG-Dairy Data App

The AADGG-Dairy Data App demonstrates robust technical architecture with real-time cloud-based data synchronization, enabling comprehensive livestock performance tracking across multiple countries and generating the largest body of dairy performance data from smallholder systems in East Africa. The system successfully implements genetic profiling and ranking algorithms that identify top-performing dairy bulls and cows based on efficiency metrics, providing evidence-based breeding recommendations for artificial insemination programs. The platform benefits from strong institutional backing through ILRI and the Bill and Melinda Gates Foundation, ensuring sustained technical support, regular updates, and integration with national livestock development initiatives. The mobile interface accommodates diverse literacy levels

through intuitive visual designs and has demonstrated successful adoption across varied user groups including farmers, AI technicians, and extension officers.

National Animal Identification and Traceability System (NAITS)

NAITS provides essential infrastructure for systematic animal identification and registration, addressing the critical weakness that Rwanda previously lacked a comprehensive animal identification system. The centralized database enables effective disease surveillance, movement tracking for epidemic control, and verification of animal ownership for trade and insurance purposes. Government ownership ensures long-term sustainability and potential for mandatory adoption across all livestock keepers. The system establishes standardized identification protocols using ear tags and unique identifiers that create foundational infrastructure upon which other genetic management systems can build. Integration with veterinary service delivery systems enables efficient resource allocation and targeted interventions during disease outbreaks.

Community-Based Breeding Program (CBBP) Systems

CBBP systems demonstrate successful participatory approaches where farmers' needs, views, and decisions guide breeding objectives, with programs covering 3,200 households showing measurable improvements in birth weight, weaning weight, and lambing intervals that translate to improved household livelihoods. The record-keeping tools, whether paper-based or digital, are designed through co-creation with farming communities, ensuring cultural appropriateness and alignment with local practices. Community animal health workers and breeding coordinators provide ongoing technical support, creating sustainable capacity within villages. The cooperative structure facilitates collective action, shared learning, and peer-to-peer knowledge transfer that reinforces adoption of improved practices. Performance data collected

through these systems has enabled research breakthroughs in identifying optimal breed compositions for local environments.

2.5.2 Weaknesses of Existing Systems

AADGG-Dairy Data App

The system's main limitation is its exclusive focus on artificial insemination, making it unsuitable for the approximately 75% of cattle farmers who rely on natural mating. Limited availability of AI providers and high service costs relative to farm-gate milk prices further restrict access for resource-constrained smallholders. The AADGG app does not support inbreeding coefficient calculation for natural matings, provide breeding recommendations based on locally available bulls, or facilitate farmer-to-farmer exchange of breeding stock. Its extensive data entry requirements create adoption barriers for farmers with limited time, literacy, or digital skills, while its emphasis on elite breeding and performance ranking assumes access to superior genetics that most smallholders cannot afford. This elite orientation, combined with limited local language support and a design centered on intensive dairy production workflows, results in a clear mismatch between the system's capabilities and the practical breeding decisions of smallholder farmers.

National Animal Identification and Traceability System (NAITS)

NAITS suffers from a fundamental mismatch between its administrative design and farmers' genetic management needs. The system provides no breeding decision support, inbreeding calculations, or pedigree analysis capabilities that would inform mating choices. Access is restricted to government officials and veterinary personnel through desktop interfaces,

with no farmer-facing mobile application for direct engagement. Pedigree recording is minimal or absent, capturing only basic identification without the multi-generational lineage data necessary for genetic analysis. Performance data collection (milk yields, growth rates, reproductive efficiency) is not integrated, preventing evidence-based breeding decisions. The system operates primarily as a livestock census and movement tracking tool rather than a genetic improvement platform, offering farmers no practical guidance on which animals should be bred together to optimize offspring quality while avoiding inbreeding depression.

Community-Based Breeding Program (CBBP) Systems

CBBP systems face critical challenges managing inbreeding within member households due to few animals per household and lack of systematic tracking of genetic relationships, with expansion of interbreeding occurring as improved sires are distributed across breeding tracts of different breeds. The digital tools are typically designed for data collection by program staff rather than direct farmer use, creating dependency on external facilitators and limiting farmer autonomy in decision-making. Automated breeding recommendation algorithms are absent—data is collected but not processed into actionable guidance. The systems lack scalability beyond organized program participants, excluding independent farmers outside cooperative structures who represent the majority of livestock keepers. Cross-community networking features that would enable genetic exchange beyond immediate program boundaries are not implemented. Sustainability concerns arise when donor funding ends, as systems are often program-specific rather than commercially viable standalone products.

2.5.3 Comparison of this system with current tools

The proposed solution fundamentally differs by targeting the majority of farmers practicing natural mating with locally available breeding stock. Instead of requiring genomic testing, the application employs computer vision-based machine learning—a multi-task CNN for breed composition estimation and a Siamese network for genetic relatedness detection—enabling farmers to make informed breeding decisions using only smartphone photos. While AADGG connects farmers to centralized AI services, the proposed system creates decentralized farmer-to-farmer breeding networks, allowing smallholders to discover and access suitable breeding animals within their communities. The hybrid recommendation engine combines genetic rules with collaborative filtering that learns from actual breeding outcomes, providing progressively smarter suggestions as the user base grows. Critically, the proposed solution operates offline-first with on-device ML inference, whereas AADGG requires connectivity for core functionality. These systems serve complementary market segments: AADGG for the AI-accessible minority, the proposed application for the natural-mating majority.

The proposed solution addresses a fundamentally different problem space. While NAITS asks "which animals exist and where are they?", the proposed application asks "which animals should breed together to produce optimal offspring?" The proposed system incorporates three distinct ML models—breed composition estimation, relatedness detection, and collaborative filtering recommendations—providing actionable breeding guidance rather than administrative records. Unlike NAITS's restriction to government personnel, the proposed application empowers individual farmers with direct mobile access to genetic management tools. However, the systems are potentially complementary: future integration could leverage NAITS identification infrastructure while adding the proposed solution's breeding intelligence layer. The

proposed application also implements prospective pedigree building, automatically constructing genetic lineages as farmers record breeding events over time, creating the multi-generational relationship data that NAITS currently lacks. This forward-looking data collection transforms every breeding event into both an immediate decision point and a contribution to the growing genetic knowledge base.

Rather than depending on program facilitators for data entry, the application puts genetic management tools directly in farmers' hands through intuitive mobile interfaces. The Siamese network for relatedness estimation solves the inbreeding tracking challenge by detecting genetic relationships from photos and metadata even when complete pedigrees don't exist. The collaborative filtering recommendation engine creates exactly the cross-community breeding networks that CBBPs lack, connecting farmers with suitable breeding animals across geographic and organizational boundaries. Critically, the proposed system operates as a scalable software platform accessible to any farmer with a smartphone, not just members of organized programs, dramatically expanding potential reach. The application essentially digitizes and democratizes the CBBP model: maintaining community-driven breeding objectives and participatory decision-making while adding ML-powered intelligence, systematic pedigree tracking, and network effects that individual CBBPs cannot achieve. As farmers record breedings prospectively, the system builds both individual pedigrees and collective knowledge about which breeding strategies succeed in different contexts, creating a self-reinforcing improvement cycle.

2.6 Code of Ethics and Considerations

This research adheres to Rwanda's Law on Data Protection and Privacy (Law N° 058/2021) and the Nagoya Protocol on Access to Genetic Resources ratified by Rwanda in 2016 (AU-IBAR, 2016), ensuring all activities prioritize farmer well-being and data sovereignty. All pilot

participants will provide written informed consent explaining data collection purposes (animal photos, breeding records, outcome ratings), voluntary participation with withdrawal rights, and data usage for ML model training and research publications. Personal data collection is minimized to essential information only . Anonymization protocols remove personal identifiers before any data sharing, with geographic aggregation to district level and exclusion of small cohorts preventing re-identification. The system implements role-based access controls where farmers access only their own data, while ML training pipelines access only anonymized datasets . The recommendation system incorporates explainable AI showing farmers why specific bulls are suggested , maintaining human agency rather than black-box algorithmic compliance, while animal welfare constraints prevent harmful recommendations such as breeding closely related animals or size-mismatched pairs risking calving complications. Following Nagoya Protocol principles, genetic data ownership remains with Rwanda and farmer communities, with commitments to open-access publication, benefit-sharing negotiations involving RAB and farmer representatives if commercial opportunities arise, capacity building through training RAB extension officers , and post-research sustainability planning ensuring application maintenance transitions to local institutions rather than abrupt termination that would harm farmers dependent on the system. Continuous ethical oversight includes quarterly review meetings with stakeholders, adverse event protocols, and independent monitoring for algorithmic bias or unintended consequences, with transparent reporting of all ethical concerns in research publications maintaining accountability to responsible innovation principles.

2.7 Summary

System	Strengths	Weaknesses
AADGG-Dairy Data App	• Demonstrates robust technical architecture with real-time cloud-based data synchronization • The system successfully implements genetic profiling and ranking algorithms that identify top-performing dairy bulls and cows based on efficiency metrics • The platform benefits from strong institutional backing through ILRI and the Bill and Melinda Gates Foundation, ensuring sustained technical support, regular updates, and integration with national livestock development initiatives. • The mobile interface accommodates diverse literacy levels through intuitive visual designs and has demonstrated successful adoption across varied user groups	• The system's main limitation is its exclusive focus on artificial insemination, making it unsuitable for the approximately 75% of cattle farmers who rely on natural mating. • Limited availability of AI providers and high service costs relative to farm-gate milk prices further restrict access for resource-constrained smallholders. • The AADGG app does not support inbreeding coefficient calculation for natural matings, provide breeding recommendations based on locally available bulls, or facilitate farmer-to-farmer exchange of breeding stock. • Its extensive data entry requirements create adoption barriers for farmers with limited time, literacy, or digital skills

		• Emphasis on elite breeding and performance ranking assumes access to superior genetics that most smallholders cannot afford.
National Animal Identification and Traceability System (NAITS)	• The centralized database enables effective disease surveillance, movement tracking for epidemic control, and verification of animal ownership for trade and insurance purposes . • Access is restricted to government officials and veterinary personnel through desktop interfaces, with no farmer-facing mobile application for direct engagement. • Government ownership ensures long-term sustainability and potential for mandatory adoption across all livestock keepers. • The system establishes standardized identification protocols using ear tags and unique	• Pedigree recording is minimal or absent, capturing only basic identification without the multi-generational lineage data necessary for genetic analysis. • Access is restricted to government officials and veterinary personnel through desktop interfaces, with no farmer-facing mobile application for direct engagement. • Performance data collection (milk yields, growth rates, reproductive efficiency) is not integrated, preventing evidence-based breeding decisions. • The system operates primarily as a livestock census and movement tracking tool rather than a genetic improvement platform

	identifiers that create foundational infrastructure upon which other genetic management systems can build.	
Community-Based Breeding Program (CBBP) Systems	• The record-keeping tools, whether paper-based or digital, are designed through co-creation with farming communities, ensuring cultural appropriateness and alignment with local practices. • Performance data collected through these systems has enabled research breakthroughs in identifying optimal breed compositions for local environments. • The cooperative structure facilitates collective action, shared learning, and peer-to-peer knowledge transfer that reinforces adoption of improved practices. • Community animal health workers and	• CBBP systems face critical challenges managing inbreeding within member households due to few animals per household and lack of systematic tracking of genetic relationships, with expansion of interbreeding occurring as improved sires are distributed across breeding tracts of different breeds. • The digital tools are typically designed for data collection by program staff rather than direct farmer use, creating dependency on external facilitators and limiting farmer autonomy in decision-making. • Automated breeding recommendation algorithms are absent—data is collected but not

	breeding coordinators provide ongoing technical support, creating sustainable capacity within villages. •	processed into actionable guidance. • The systems lack scalability beyond organized program participants, excluding independent farmers outside cooperative structures who represent the majority of livestock keepers. • Cross-community networking features that would enable genetic exchange beyond immediate program boundaries are not implemented.

CHAPTER THREE : SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the design , analysis , and overall structure of the Multi-trait Animal Targeted Compatibility Hub (MATCH) . This describes how the system is orchestrated and organized , how components interact and also key decisional and architectural points . This chapter will demonstrate the proposed system's functional prerequisites into a scalable solution.

The development of MATCH follows the Agile development methodology , emphasizing iterative development , continuous feedback , and incremental delivery of features . This approach enables the system to evolve progressively , allowing new functionality to be introduced in small, manageable increments while building upon previously implemented features . As a result , users are able to benefit from early system releases , while development refines the system based on testing and user feedback . The proposed system architecture prioritizes real-time interaction, responsiveness, and reliability , ensuring that users can access compatibility insights and system features efficiently . Emphasis is placed on modular design to support maintainability , scalability , and future enhancements. This chapter further outlines the architectural components , data flow , and interaction mechanisms that collectively support the core objectives of MATCH .

3.2 Research Design

This research paper employs the use of quantitative research design patterns within Agile based development methodologies for objectiveness and uses statistical and factual approaches . An experimental research design pattern will be used since MATCH consists of a bunch of

multi-step and level Machine Learning models and changes in inputs and performance will be observed and measured to give the most accurate and efficient models in the solution . Agile methodologies being chosen over others like waterfall since requirements and demands from stakeholders , users and more features to be released require constant and non-stop iteration . The proposed research solution reflects on the features being implemented such as Multi-step Machine Learning models requiring trial and error .

3.3 Class Diagram

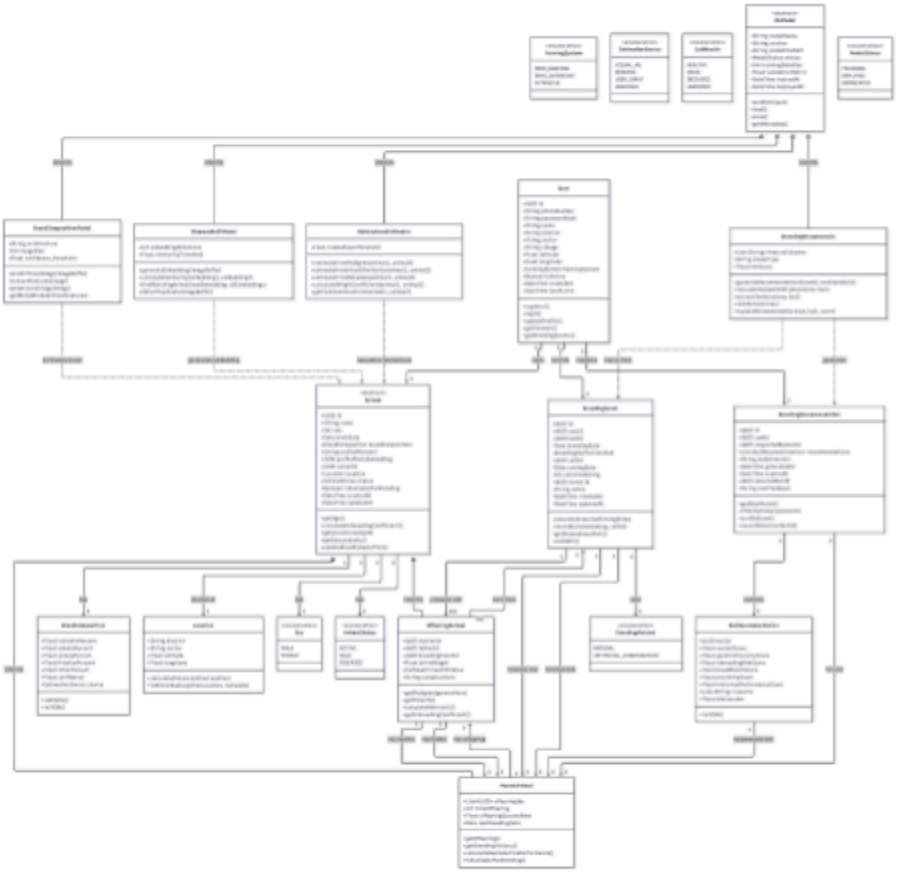


Figure 1.2 : Class Diagram

The livestock genetic matching system implements an object-oriented architecture organized into three primary layers: domain entities, value objects/enumerations, and machine learning models. The User class serves as the root entity representing farmers, containing

authentication credentials , profile information , and optional farming context . Users maintain one-to-many relationships with Animals, breeding events, and breeding recommendations, enabling them to manage their livestock portfolio, record breeding activities, and request machine learning powered breeding suggestions.

The Animal class hierarchy demonstrates classical inheritance with an abstract Animal base class that encapsulates common properties like unique identifier, name, sex, birth date, breed composition, photo url and embedding, ownership, location, availability status, and timestamps with shared behaviors like age calculation, inbreeding coefficient computation, and ancestor/descendant retrieval. This base class specializes into two concrete subclasses which are parent animal, which tracks offspring relationships through offspring IDs, maintains breeding history metrics (total offspring, success rates, last breeding date), and provides reproductive performance calculations suitable for bulls and cows actively used in breeding; and OffspringAnimal, which stores parental lineage (mother ID, father ID), links to the breeding event that produced it, records birth outcomes (weight, health status, complications), and implements pedigree-specific methods for calculating heterosis and individual inbreeding coefficients. Animals compose with value objects including BreedComposition (percentages of Holstein, Ankole, Jersey, Friesian, and other breeds with confidence scores and estimation source), Location (geographic coordinates with distance calculation utilities), and enumerations for Sex, AnimalStatus, and other categorical attributes that ensure type safety and data consistency.

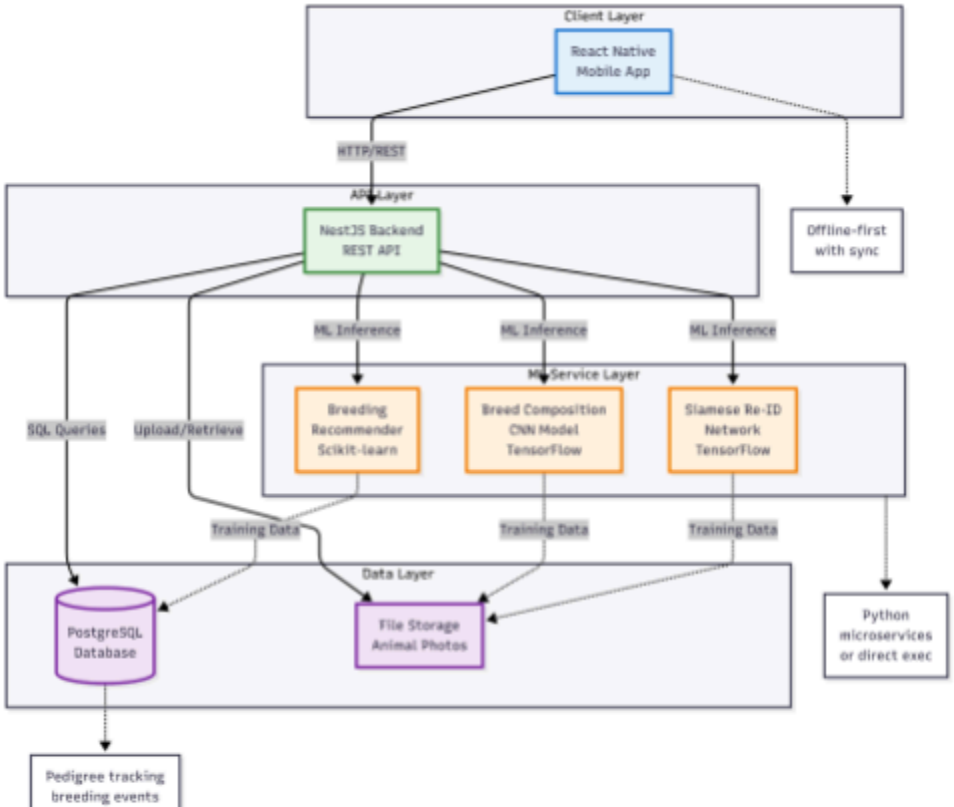
The BreedingEvent entity captures the complete breeding lifecycle, linking a cow (ParentAnimal) and bull (ParentAnimal) with breeding date, method (natural or artificial insemination via the BreedingMethod enumeration), and expected calving date, while optionally

referencing the resulting calf (OffspringAnimal) once born along with actual calving date, outcome rating (1-5 stars), and health details. This bidirectional relationship between BreedingEvent and OffspringAnimal enables automatic pedigree construction: when a calf is born and linked to its breeding event, the system automatically populates the calf's mother_id and father_id by tracing back through the breeding event to the parent animals, building multi-generational family trees prospectively without requiring farmers to manually record lineage.

The ML model layer implements a polymorphic hierarchy with an abstract MLModel base class defining common ML infrastructure—model name, version, file path, deployment status (via ModelStatus enumeration), training metadata (dataset size, validation metrics, timestamps)—and shared operations for prediction, persistence, and metadata retrieval. Four concrete ML model classes inherit from this base: BreedCompositionModel (CNN-based breed estimation from photos using transfer learning on EfficientNet architecture), SiameseReIDModel (embedding generation and similarity calculation for duplicate detection and visual relatedness estimation), RelatednessEstimator (hybrid approach combining pedigree-based Wright's coefficient calculations, visual similarity from Siamese embeddings, and metadata-based heuristics when complete information is unavailable), and BreedingRecommender (self-training model that learns from historical BreedingEvent outcomes to generate compatibility scores and ranked bull recommendations). These ML models interact with domain entities through dependency relationships: BreedCompositionModel estimates breed for Animals, SiameseReIDModel generates embeddings for Animals, RelatednessEstimator calculates genetic relationships between Animal pairs, and BreedingRecommender learns from BreedingEvents to generate BreedingRecommendations.

The BreedingRecommendation aggregate encapsulates the ML-powered recommendation workflow, containing a collection of BullRecommendation value objects that each represent a single bull candidate with explainable scoring components (overall compatibility score decomposed into genetic diversity, inbreeding risk, breed match, geographic proximity, and historical performance subscores) along with reasoning explanations and distance metrics. Each BreedingRecommendation associates with a specific cow (ParentAnimal) requested by a User, tracks the ML model version that generated it, includes expiration timestamps for cache invalidation, and captures user feedback (which bull was actually selected, textual feedback) that feeds back into the BreedingRecommender's self-training pipeline, completing the continuous learning loop. This architecture demonstrates SOLID principles with clear separation of concerns: domain logic resides in entity classes, ML inference is isolated in model classes with dependency injection patterns, value objects ensure immutability for breed composition and location data, and enumerations provide type-safe categorical constraints throughout the system.

3.4 System Architecture



The livestock genetic matching system implements a four-tier architecture that separates presentation, business logic, machine learning intelligence, and data persistence into distinct layers with clear interfaces and responsibilities. The Client Layer consists of a React Native mobile application that provides the farmer-facing user interface, implementing an offline-first architecture with local data persistence and background synchronization to ensure uninterrupted functionality in Rwanda's rural areas with intermittent network connectivity. Farmers interact exclusively through this mobile app to register animals, capture photos, record breeding events, view pedigrees, and request breeding recommendations, with all user actions queued locally when offline and automatically synced to the backend when connectivity is restored.

The API Layer features a NestJS backend that serves as the central orchestration point, exposing REST endpoints consumed by the mobile client for all CRUD operations (create animals, retrieve breeding history, update user profiles, delete records). This backend handles authentication, request validation, business logic enforcement (such as preventing breeding between animals with high relatedness scores), and coordinates communication between the client, machine learning services, and data layer. NestJS routes incoming requests to appropriate services: simple queries are forwarded directly to the PostgreSQL database via SQL, photo uploads are routed to file storage, and machine learning dependent operations trigger inference calls to the specialized machine learning models.

The machine learning Service Layer houses three independent machine learning models implemented as Python-based microservices (or direct execution modules) that the NestJS backend invokes for inference: the Breed Composition CNN (TensorFlow-based EfficientNet model that analyzes animal photos to estimate Holstein, Ankole, Jersey, Friesian, and other breed percentages), the Siamese Re-ID Network (TensorFlow neural network generating 256-dimensional embeddings from photos to detect duplicate registrations and calculate visual similarity as a proxy for genetic relatedness), and the Breeding Recommender (Scikit-learn gradient boosting model that learns from historical breeding outcomes to generate compatibility scores and ranked bull suggestions). These machine learning models maintain bidirectional relationships with the data layer: they consume training data from the PostgreSQL database (breeding events, outcomes, user feedback) and file storage (animal photos with genomic labels) during periodic retraining cycles to continuously improve accuracy through self-supervised learning.

The Data Layer provides persistent storage through two complementary systems: a PostgreSQL database managing structured relational data (user accounts, animal registries with pedigree linkages via self-referencing foreign keys, breeding events that automatically build family trees, machine learning model versions, and recommendation caches), and File Storage (Supabase S3) housing unstructured binary data (animal photos, trained ML model artifacts as .tflite and .pkl files, and model training datasets). The database implements the complete domain model with tables for users, animals (with mother_id/father_id enabling multi-generational pedigree queries), breeding events (linking cows, bulls, and offspring to construct lineages prospectively), and Machine learning metadata (tracking model versions, training metrics, and deployment status).

3.4.1 Machine Learning Ops Diagram

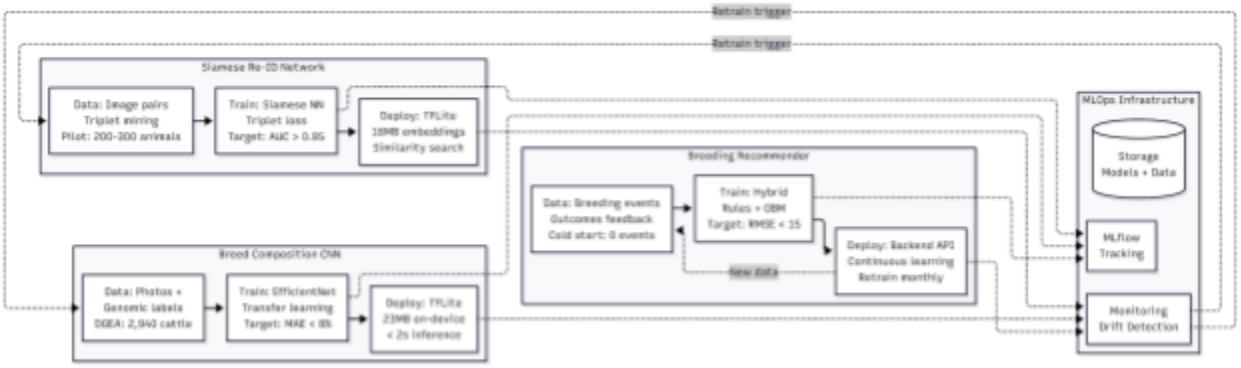


Figure 1.9 : MLOps Diagram

The integrated workflow begins with the Breed Composition CNN, where the ResNet50 model serves as the "eyes" of the system. In Phase A, the model utilizes its frozen pre-trained weights to identify broad livestock categories, while Phase B refines these into 14 specific breed signatures. This classification logic is then exported as a lightweight TFLite model, enabling real-time, on-device inference for farmers to instantly verify the genetic makeup of their livestock with high precision.

Moving from broad breeds to specific individuals, the Siamese Re-ID Network repurposes the ResNet50 backbone as a feature extractor. By processing image pairs through identical "twin" branches, the system generates unique biometric embeddings that identify specific animals within a herd. Finally, these outputs feed into the Breeding Recommender, a self-training hybrid system that combines the verified breed and identity data with outcomes from past breeding events. This model uses a continuous feedback loop to pseudo-label new data and refine its recommendations, evolving into a sophisticated decision-support tool that advises on optimal breeding strategies for long-term herd improvement.

3.5 UML Diagrams

3.5.1 Sequence Diagram

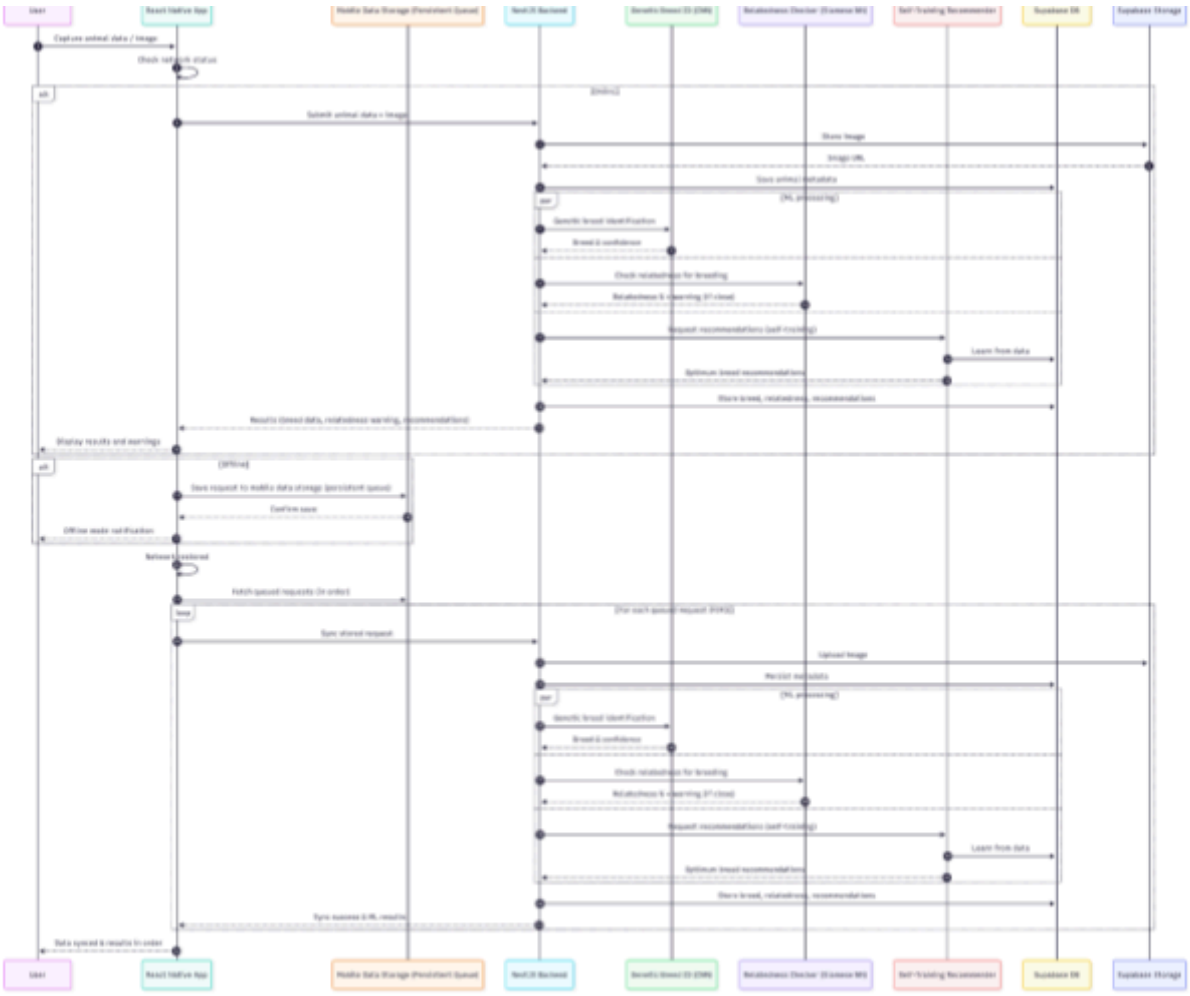


Figure 1.4 : Sequence Diagram

The livestock genetic matching system operates through three distinct workflows—online, offline, and synchronization—ensuring farmers can register animals and receive breeding recommendations regardless of network connectivity. When a farmer captures animal data and photos through the React Native mobile app, the system first checks network availability. In the online flow, the app immediately submits data to the NestJS backend API, which stores the photo in Supabase Storage and saves animal metadata to the Supabase database.

The backend then triggers three parallel machine learning processing pipelines i.e the CNN model performs genetic breed identification and returns breed composition percentages with confidence scores; the Siamese neural network checks relatedness between the new animal and existing livestock, flagging potential inbreeding risks if animals are closely related; and the self-training recommender model queries the database to learn from historical breeding outcomes and generates optimized breeding recommendations. All machine learning results are stored in the database and returned to the mobile app for immediate display to the farmer.

When network connectivity is unavailable, the system seamlessly switches to offline mode: the React Native app saves the registration request to persistent mobile storage using a simple SQLite offline database and a FIFO data structure, confirms the save to the user, and displays an offline mode notification. The farmer can continue registering multiple animals without interruption, with all data safely queued and stored locally. Once network connectivity is restored, the synchronization flow automatically activates: the app retrieves all queued requests in chronological order and processes them sequentially, uploading each animal's photo to Supabase Storage, persisting metadata to the database, and triggering the same parallel machine learning processing that would have occurred in real-time. The backend processes each queued request completely before moving to the next, ensuring data consistency and proper pedigree linkage, then confirms successful synchronization back to the mobile app, which displays all accumulated machine learning results to the farmer in the order animals were originally registered. This offline-first architecture ensures uninterrupted farmer productivity in Rwanda's rural areas with intermittent connectivity, while maintaining full machine learning powered functionality and data integrity once connection is reestablished.

3.5.2 ERD Diagram

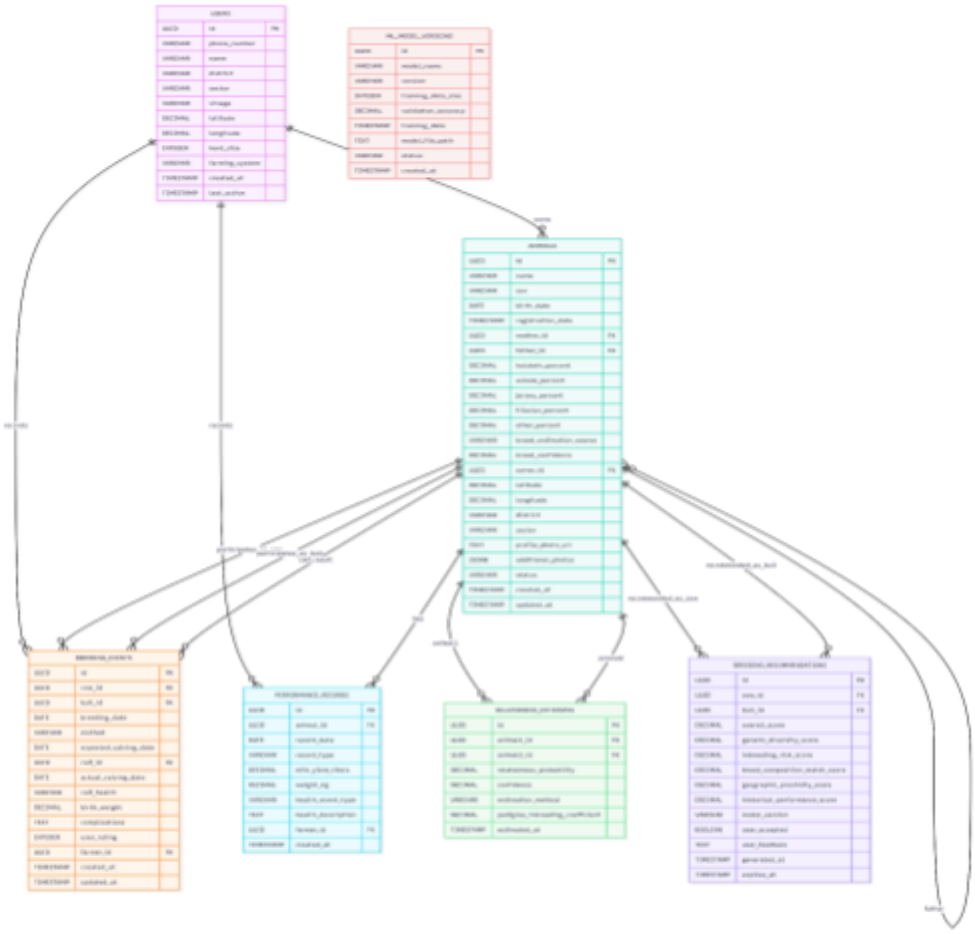


Figure 1.3 : ERD Diagram

The livestock genetic matching system database is built around seven core entities that work together to enable Machine Learning powered breeding recommendations and pedigree tracking. At the center is the users entity, representing farmers (users basically) who register accounts with their contact information (phone number, name), geographic location (district, sector, village with coordinates), and optionally farming context (herd size and farming system type). Each user owns multiple entries in the animals entity, which serves as the comprehensive registry of all livestock in the system. Animals are uniquely identified and contain essential genetic information including breed composition percentages , estimated by machine learning

models and/or user input , along with confidence scores and estimation sources. Critically, the animal entity implements a self-referential relationship through mother_id and father_id foreign keys, enabling automatic pedigree tree construction across multiple generations as breeding events are recorded over time.

The breeding events entity captures the complete lifecycle of breeding activities, linking female livestock and male livestock (both foreign keys to animals) with breeding dates, methods (natural or artificial insemination), and expected calving dates. When offspring are born, the calf_id links back to animals, automatically establishing parent-child relationships that build the pedigree prospectively. Each breeding event can be rated by farmers (1-5 stars) and includes outcome details such as calf health status, birth weight, and any complications—this feedback data feeds directly into the self-training recommendation model, creating a continuous learning loop. The performance_records entity optionally tracks milk yields, weights, and health events for animals, though this is not core to the genetic matching functionality and serves primarily as additional data for recommendation model training.

Two entities support the Machine learning powered recommendation engine: relatedness_estimates stores calculated genetic relationships between animal pairs, using either pedigree-based Wright's coefficient calculations when lineage data exists, visual similarity scores from the Siamese network when photos are available, or metadata-based heuristics as fallback and supplemental to the network. This prevents inbreeding by flagging potentially related animals before breeding decisions are made. The breeding events entity caches Machine learning generated breeding suggestions, storing compatibility scores broken down into explainable components (genetic diversity, inbreeding risk, breed match, proximity, and historical

performance), along with the model version that generated them and user feedback on whether recommendations were accepted thus , enabling A/B testing and model improvement tracking.

Finally, the ml_versions_entity entity provides governance and versioning for the machine learning pipeline, tracking each deployed model iteration (breed composition CNN, Siamese re-identification network, and breeding recommender) with metadata including training data size, validation accuracy metrics, deployment status, and file paths to model artifacts. This enables complete model lineage tracking, rollback capabilities if performance degrades, and reproducibility of results. Together, these seven entities create a complete ecosystem where user-generated data (animal registrations, breeding events, outcomes) continuously feeds Machine learning models that provide increasingly intelligent breeding recommendations, while pedigree relationships build automatically in the background, solving the fundamental challenge that Rwandan farmers are facing , managing genetics without historical pedigree records.

3.5.3 Use Case Diagram

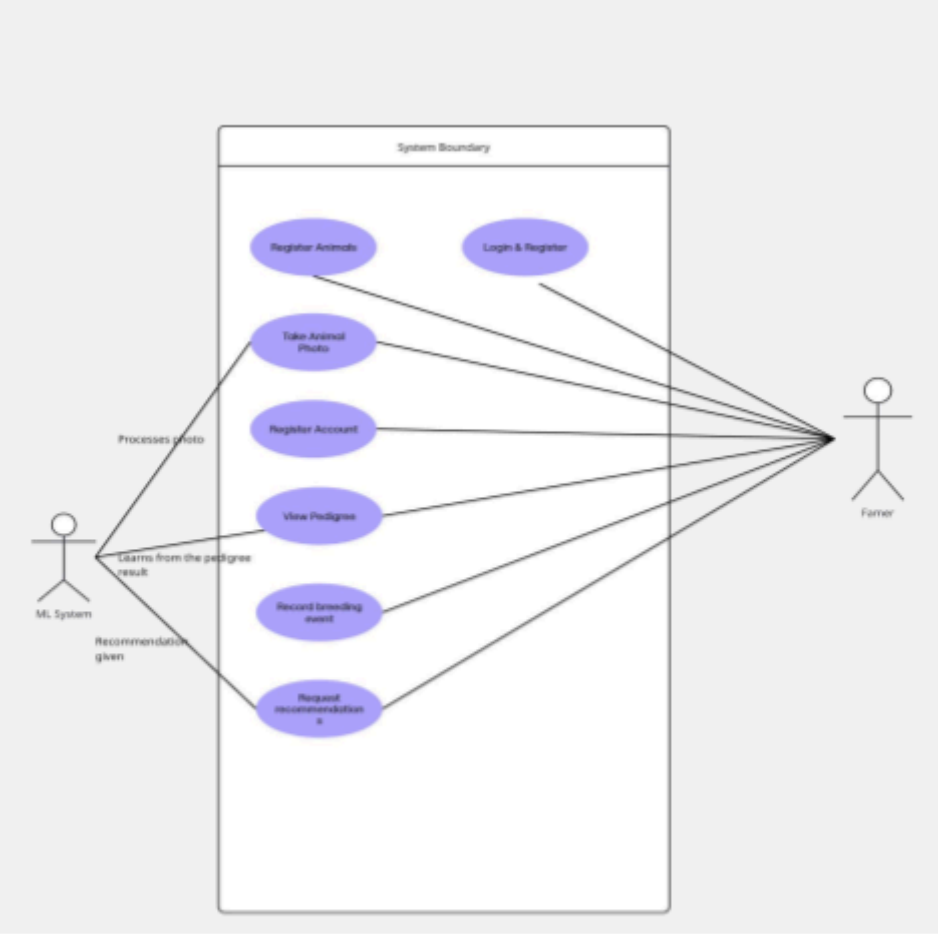


Fig : Use Case Diagram

The livestock genetic matching system centers around the Farmer as the primary actor who interacts with the application through nine core use cases organized into three functional domains i.e account management, animal management, and breeding decision support. The farmer journey begins with Register Account and Login for authentication, establishing their identity and access to the system. Once authenticated, farmers engage with animal management use cases including Register Animal , Take Animal Photo (captured during registration to enable machine learning powered features), and View Pedigree (visualizing family tree relationships built automatically from breeding records). The photo upload triggers three parallel machine

learning processing workflows handled by an external machine learning System actor . The breeding management domain encompasses Record Breeding Event (documenting when a cow and bull mate, which automatically builds pedigree lineages when offspring are born), View Bulls (browsing available male animals in the farmer's area for potential breeding), and Request Recommendations (the system's flagship machine learning powered feature). When a farmer requests breeding recommendations for their cow, the machine learning system processes this through the recommendation engine , which applies a self-training machine learning model that has learned from historical breeding outcomes across all farmers in the system. This recommender analyzes the cow's genetics, location, and production system, evaluates available bulls for compatibility (considering breed composition match, inbreeding risk, geographic proximity, and historical breeding success), and returns ranked matches with explainable reasoning for each suggestion. The farmer can then review these recommendations, contact bull owners, record breeding events, and later provide outcome feedback (whether the calf was healthy, growth performance, etc.)—this feedback loops back into the machine learning system to continuously improve recommendation accuracy through self-training, creating an intelligent system that becomes progressively smarter as more farmers use it and more breeding outcomes are recorded over time.

3.6 Development Tools

Category	Tool	Reason
Frontend	Frontend/Mobile	Ease of use , not system intensive , builds for both iOs and Android , Auto

		updates after builds and release
Backend	NestJS	Scalability , No tight coupling , Easy testing , a lot of abstraction
Containerization & Local Testing	Docker	Standard tool for containerization , Many images helping in testing using different scenarios , easy to use
Offline and Synchronization Queue	BullMQ	Retry capability, cron job features , rollback and metric capabilities , acknowledgement
Testing and Automation	Github Actions	Standard automation running workflows , Access to actions and reacting to events , builds and tests for any OS using bots
API Testing	Insomnia/Postman	Standard tool , many testing protocols available ,

		groups routes and endpoints for reusability
Load Testing and monitoring	Grafana K6 & Grafana tools , Sentry	Reports directly to platform of choice , free and easy use , nice report GUI available
Database and S3 Storage	Supabase and Supabase s3	Ease of use , free tiers available with s3 storage, frontend authentication , connected to other tools and apps , No need for VM or server setup

CHAPTER 4: SYSTEM IMPLEMENTATION AND TESTING

4.1 Implementation and Coding

4.1.1 Introduction

Chapter Three defined what MATCH should do; this chapter shows how it was built. Two codebases emerged from that design: a NestJS server-side API integrated with a Python ML scoring microservice, and a React Native mobile client. The account that follows concerns the enacted choices—which libraries were selected, how the principal modules were structured, and what the core algorithms look like in source code. Development proceeded across four Agile sprints, each producing a testable increment verified against the functional requirements before the next sprint began.

4.1.2 Description of Implementation Tools and Technology

Two constraints shaped every technology decision. On the server side, the Render free-tier container imposes a hard 512 MB RAM ceiling. On the client side, core genetic-matching functionality had to run on low-cost Android smartphones in areas of intermittent connectivity. Table 1 lists each tool used in the final implementation, organised by architectural layer.

Layer	Tool / Library	Version	Role in MATCH
Backend API	NestJS	11.0.1	Modular REST framework; TypeScript-native dependency injection and testability
Backend API	TypeScript	5.7.3	Static typing across all server-side modules and DTOs

ORM	Prisma	7.4.0	Type-safe PostgreSQL client with schema-driven migrations
Database	PostgreSQL (Supabase)	Latest	Relational store for users, animals, breeding records, and recommendations
File Storage	Supabase S3	Latest	Animal photo uploads and trained ONNX model artifact storage
Authentication	JWT + Passport + argon2	11.x	Stateless bearer tokens; argon2 password hashing
ML Runtime (server)	ONNX Runtime (Python)	1.x	Lightweight inference (~80 MB RAM vs. ~400 MB for TensorFlow-CPU)
ML Training	TensorFlow / Keras	2.x	MobileNetV2-based BreedingRecommender training and tf2onnx export
ML On-Device	react-native-fast-tflite	Latest	TFLite inference with CoreML (iOS) and GPU (Android) hardware delegates
Containerisation	Docker (multi-stage)	Latest	Node 20-slim builder stage + Python 3 venv runtime stage in one image
Monitoring	Prometheus + Grafana	Latest	Per-route request counters and latency histograms via custom middleware
API Documentation	Swagger / OpenAPI	5.0.1	Auto-generated endpoint reference at /api/docs with JWT bearer auth
Mobile Framework	React Native + Expo SDK	~54	Single TypeScript codebase compiled to iOS and Android
Navigation	Expo Router	4.x	File-based screen routing across all application views

Secure Storage	Expo SecureStore	Latest	Hardware-backed AES-encrypted JWT and session data storage
Location Services	Expo Location	Latest	GPS coordinates captured at signup for proximity-based bull discovery
HTTP Client	Axios	Latest	Request interceptors for automatic JWT injection; 401 auto-logout
Unit / E2E Testing	Jest + ts-jest + Supertest	30.x	Service-level unit tests and HTTP-level end-to-end tests
Caching / Messaging	Redis (ioredis)	5.x	In-memory session store for WebSocket user–socket mapping;
Real-Time Messaging	Socket.IO (via NestJS WebSocketGateway)	4.x	Bidirectional farmer-to-farmer messaging gateway on port 8000 with JWT authentication
Load Testing	Grafana k6	Latest	Concurrent-user stress testing of REST endpoints

4.1.3 Backend Module Architecture

Ten feature modules form the backbone of the NestJS backend, each encapsulating a controller, service, and data-transfer object (DTO) layer for a single domain. The root AppModule registers all eight, wires the global ConfigModule for environment variable access, and attaches a custom Prometheus metrics middleware to every incoming route. At startup, main.ts bootstraps the application instance, mounts cookie-parser, and configures the Swagger portal at /api/docs—complete with JWT bearer authentication—before binding to the PORT

environment variable (default: 3000). Table 2 summarises each module's domain responsibility and its public API surface.

Module	Domain Responsibility	Primary Endpoints
UsersModule	Farmer profile creation, retrieval, and update; PostGIS location storage	POST /users, GET /users/:id, PATCH /users/:id
AnimalsModule	Livestock registration, Supabase photo upload, breed profile, parent-child pedigree linkage	POST /animals, GET /animals, GET /animals/:id, PATCH /animals/:id
BreedingEventsModule	Breeding event recording, outcome rating, calf linkage, automated pedigree chain construction	POST /breeding-events, GET /breeding-events/:id, PATCH /breeding-events/:id
RecommendationsModule	ML-powered pair scoring, seven-day result caching, proximity-based quick-matches via PostGIS, acceptance and animal-locking transactional workflow	GET /recommendations, GET /recommendations/quick-matches, PATCH /recommendations/:id/accept, PATCH /recommendations/:id
AuthModule	JWT login and signup, JwtStrategy guard, bcrypt/argon2 credential verification	POST /auth/signup, POST /auth/login, GET /auth/me
StorageModule	Supabase S3 photo upload orchestration and signed URL retrieval	POST /storage/upload
MessagesModule	WebSocket gateway (AppGateway) for real-time farmer-to-farmer messaging; JWT authentication on connect; Redis-backed socket–user session mapping	WebSocket port 8000 (msgToServer / msgToClient events)

RedisModule	ioredis client provider (REDIS_CLIENT injection token); exported Redis service for use by MessagesModule and other consumers	GET /redis/ping (health-check)
JwtserviceModule	Singleton JWT sign/verify service shared across Auth and Guard modules	(Internal — no public endpoints)
PrismaServiceModule	Singleton Prisma client injected as a provider across all feature modules	(Internal — no public endpoints)

Sample Code 4.1 — Root module registration (src/app.module.ts)

Sample Code 4.1 shows the updated AppModule class. In addition to the original eight feature modules, it now registers MessagesModule (the WebSocket gateway) and RedisModule (the ioredis client provider). PrometheusModule registers default Node.js runtime metrics alongside a custom request counter; ConfigModule propagates all .env values globally.

```
// src/app.module.ts



import { MiddlewareConsumer, Module } from '@nestjs/common';
import { AnimalsModule } from './animals/animals.module';
import { BreedingEventsModule } from './breeding_events/breeding_events.module';
import { RecommendationsModule } from './recommendations/recommendations.module';
import { AuthModule } from './auth/auth.module';
```

```
import { MessagesModule } from './messages/messages.module';
import { RedisModule } from './redis/redis.module';
import { ConfigModule } from '@nestjs/config';
import { PrometheusModule, makeCounterProvider } from '@willsoto/nestjs-prometheus';
import { CustomMetricsMiddleware } from './middleware/grafana.middleware';


@Module({
  imports: [
    UsersModule, AnimalsModule, BreedingEventsModule,
    PrismaServiceModule, JwtserviceModule, AuthModule,
    StorageModule, RecommendationsModule,
    MessagesModule, RedisModule,
    ConfigModule.forRoot({ isGlobal: true, envFilePath: '.env' }),
    PrometheusModule.register({ defaultMetrics: { enabled: true } }),
  ],
  providers: [ AppService, makeCounterProvider({
    name: 'count', help: 'metric_help',
    labelNames: ['method', 'origin'] as string[],
  })],
})


```

```
export class AppModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(CustomMetricsMiddleware).forRoutes('*');
  }
```

```
}
```

4.1.4 Machine Learning Implementation

At request time, the NestJS RecommendationsService spawns a Python subprocess to run the ML scoring pipeline. The BreedingRecommender was trained with TensorFlow 2.x and Keras on a MobileNetV2 backbone (224 × 224 × 3 pixel input, ImageNet-pretrained, top layers removed), then exported to ONNX format via tf2onnx. Substituting ONNX Runtime (80 MB RAM) for TensorFlow-CPU (400 MB) at inference time was the critical decision that kept total memory consumption within the 512 MB container budget, with 128 MB reserved for the Node.js process

Three sub-networks compose the BreedingRecommender. A frozen MobileNetV2 feature extractor generates a 1,280-dimensional visual feature vector from each input image. A projector network concatenates that vector with a 32-dimensional learned species embedding—one of four classes: cattle, goats, pigs, and sheep then projects the combined representation to a 256-dimensional L2-normalised embedding space through two dense layers with GELU activations, layer normalisation, and 35% dropout. Finally, a score-heads network receives the 512-dimensional concatenated embedding pair for a candidate bull and cow, producing two sigmoid-activated scores: genetic diversity and breed-composition match. The overall compatibility score is: 0.40 × diversity + 0.35 × (1 − inbreeding_risk) + 0.25 × breed_match. Crucially, inbreeding risk is not predicted by the network at all; it is Wright's coefficient drawn

directly from the relatedness_estimates database table and passed into the formula unchanged, ensuring recorded pedigree data always overrides visual estimates.

Sample Code 4.2 — BreedingRecommender model definition (ML_model/export_to_onnx.py, excerpt)

```
# ML_model/export_to_onnx.py — model architecture


def build_projector(backbone_dim, embed_dim=256):
    visual_in  = keras.Input(shape=(backbone_dim,), name="visual_feat")
    species_in = keras.Input(shape=(1,), dtype=tf.int32, name="species_id")
    sp_emb = layers.Embedding(4, 32)(species_in)   # 4 species classes
    sp_emb = layers.Flatten()(sp_emb)
    x = layers.Concatenate()([visual_in, sp_emb])
    x = layers.Dense(512)(x)
    x = layers.Activation("gelu")(x)
    x = layers.LayerNormalization()(x)
    x = layers.Dropout(0.35)(x)
    x = layers.Dense(embed_dim)(x)
    x = layers.Lambda(lambda t: tf.math.l2_normalize(t, axis=-1))(x)
    return keras.Model([visual_in, species_in], x)


class BreedingRecommender(keras.Model):
    def encode(self, image, species_id, training=False):
        visual = self.backbone(image, training=False)
        return self.projector([visual, species_id], training=training)


    def call(self, inputs, training=False):
```

```
        img_a, species_a, img_b, species_b, relatedness = inputs
        emb_a = self.encode(img_a, species_a, training=training)
        emb_b = self.encode(img_b, species_b, training=training)
        pair  = tf.concat([emb_a, emb_b], axis=-1)          # 512-dim pair
        diversity, breed_match = self.score_heads(pair, training=training)
        inbreeding = tf.clip_by_value(
             tf.cast(tf.squeeze(relatedness, -1), tf.float32), 0.0, 1.0)
        # Weighted compatibility formula
        overall = (0.40 * diversity
             + 0.35 * (1.0 - inbreeding)
             + 0.25 * breed_match)
              return tf.clip_by_value(overall, 0.0, 1.0), diversity, inbreeding,
breed_match
```

Sample Code 4.3 — Batch recommendation workflow (src/recommendations/recommendations.service.ts, excerpt)

The service follows a seven-step pipeline: it validates the requesting animal's eligibility, serves a fresh seven-day cache when one exists, builds a candidate pool filtered to opposite sex, same species type, and different farm, downloads up to 50 candidate images in parallel, invokes a single Python subprocess for batch ONNX scoring, ranks all results by overall score in descending order, and finally persists the top 10 to the breeding_rec table.

```
// src/recommendations/recommendations.service.ts (excerpt)
```

```
async getRecommendations(animalId: string) {
  // Step 1 — validate
  const animal = await this.prisma.animals.findUnique({ where: { animalId } });
  if (!animal)             throw new NotFoundException("Animal not found");
  if (!animal.recommendable)
    throw new BadRequestException("Animal not eligible for recommendations");

  // Step 2 — serve fresh cache (< 7 days, >= 10 records)
  const sevenDaysAgo = new Date(Date.now() - 7 * 24 * 60 * 60 * 1000);
  const cached = await this.prisma.breeding_rec.findMany({
    where: { animalInitial: animalId, generatedAt: { gte: sevenDaysAgo } },
    orderBy: { overall_score: "desc" },
  });
  if (cached.length >= 10) return cached;

  // Step 3 — candidate pool: opposite sex, same species type, different farm
  const candidates = await this.prisma.animals.findMany({
    where: { type: animal.type,
             sex:  animal.sex === Gender.MALE ? Gender.FEMALE : Gender.MALE,
             recommendable: true, status: AnimalStatus.ALIVE,
             ownerId: { not: animal.ownerId } },
  });
  if (candidates.length === 0) return [];

  // Step 4 — parallel photo download, cap at 50
  const downloaded = (await Promise.allSettled(
    candidates.slice(0, 50).map(c =>
      this.downloadImage(c.profilePhoto, tmpPath(c.animalId))))
```

```
  ).filter(r => r.status === "fulfilled").map(r => r.value);

  // Step 5 — single Python batch subprocess
  const scored = await this.runPythonBatch(queryImg, labelA, batchFile);

  // Step 6 — rank and persist top 10
  scored.sort((a, b) => b.overall_score - a.overall_score);
  await this.prisma.breeding_rec.createMany({
    data: scored.slice(0, 10).map(r => ({
      animalInitial: animalId,
      recommendedAnimalId: r.animalId,
      overall_score:                  r.overall_score,
      genetic_diversity_score:         r.genetic_diversity_score,
      inbreeding_risk_score:           r.inbreeding_risk_score,
      breed_composition_match_score:   r.breed_composition_match_score,
    })),
  });
}
```

4.1.5 Frontend Implementation

Built on Expo SDK 54 with Expo Router handling file-based navigation, the React Native client ships two TFLite models inside its binary: livestock_mobile_vnet_final.tflite for on-device breed composition estimation, and livestock_biometric_float16.tflite—a float16-quantised Siamese Re-ID network for individual animal identification. The

react-native-fast-tflite library loads both models using the CoreML delegate on iOS and GPU acceleration libraries on Android, as configured in app.json (Sample Code 4.4). Without those hardware delegates, sub-three-second inference on mid-range handsets would not have been achievable.

JWT tokens are persisted in Expo SecureStore, which delegates storage to the iOS Secure Enclave and Android Keystore for hardware-backed AES encryption. Once a farmer completes one credential-based login, subsequent sessions can authenticate biometrically Face ID, Touch ID, or fingerprint even with the device offline; the client reads the cached token from SecureStore and verifies it against GET /auth/me when connectivity is later available. At registration, Expo Location captures GPS coordinates and transmits them to the backend, where they are stored as a PostGIS geography(Point, 4326) field that powers the distance-sorted queries behind the interactive breeder map.

Sample Code 4.4 — Frontend Expo configuration (app.json, excerpt)

```
// app.json — Expo project configuration (excerpt)
{
  "expo": {
    "name": "Match_F",
    "version": "1.0.0",
    "plugins": [
      ["react-native-fast-tflite", {
        "enableCoreMLDelegate": true,
```

```
          "enableAndroidGpuLibraries": true
        }],
        ["expo-location", {
          "locationWhenInUsePermission":
         "Match needs your location to connect you with nearby farmers."
        }],
        "expo-secure-store",
        "expo-updates"
      ],
      "android": {
        "package": "com.anonymous.Match_F",
        "adaptiveIcon": { "backgroundColor": "#134F19" },
        "permissions": [
          "android.permission.ACCESS_COARSE_LOCATION",
          "android.permission.ACCESS_FINE_LOCATION"
        ]
      }
    }
}
```

4.2 Graphical View of the Project

4.2.1 Screenshots with Description

Each screen below was prototyped in Figma and implemented in React Native with Expo Router. The layouts map directly to the functional requirements defined in Chapter One.

Screen 1: On-Device Breed Recognition

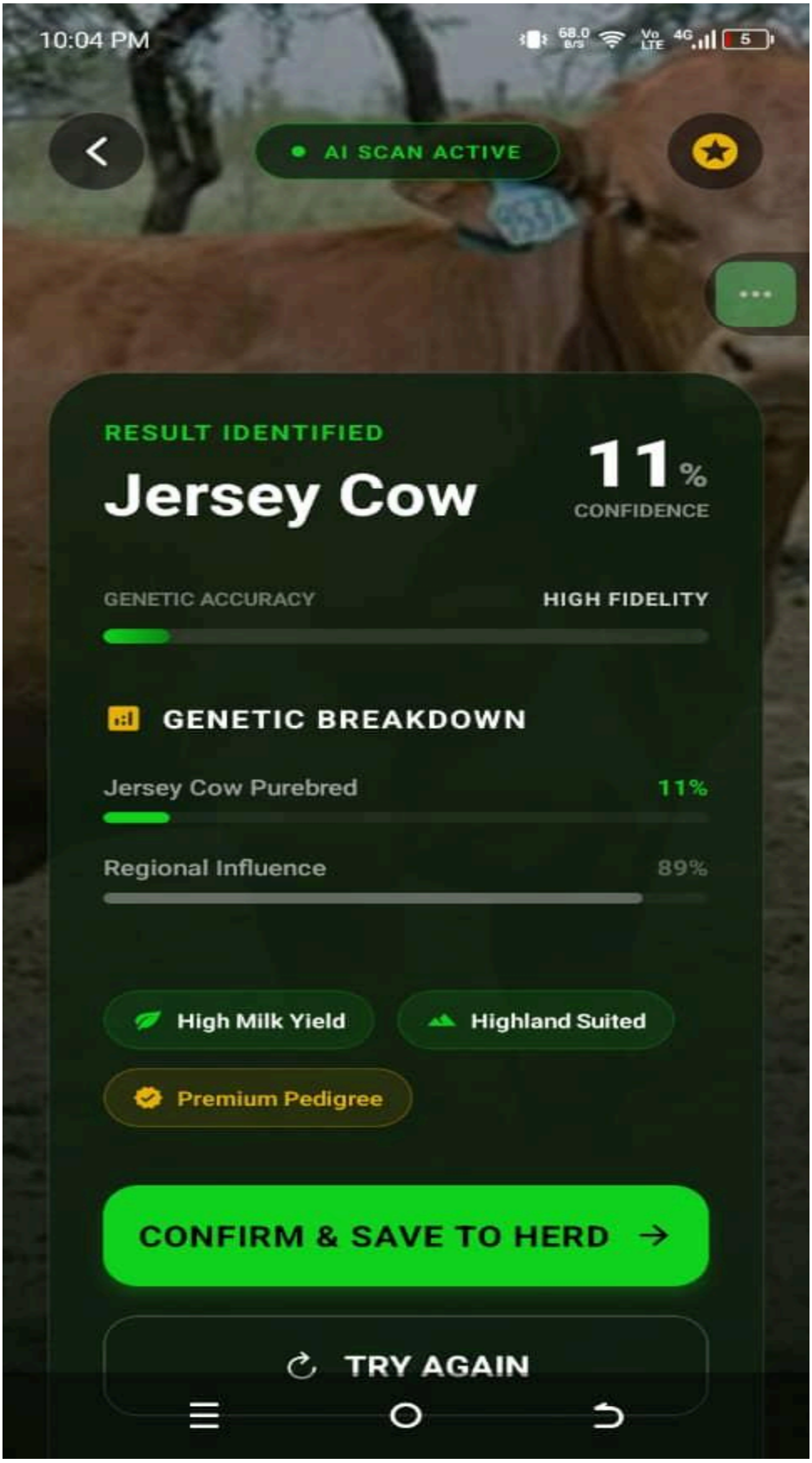
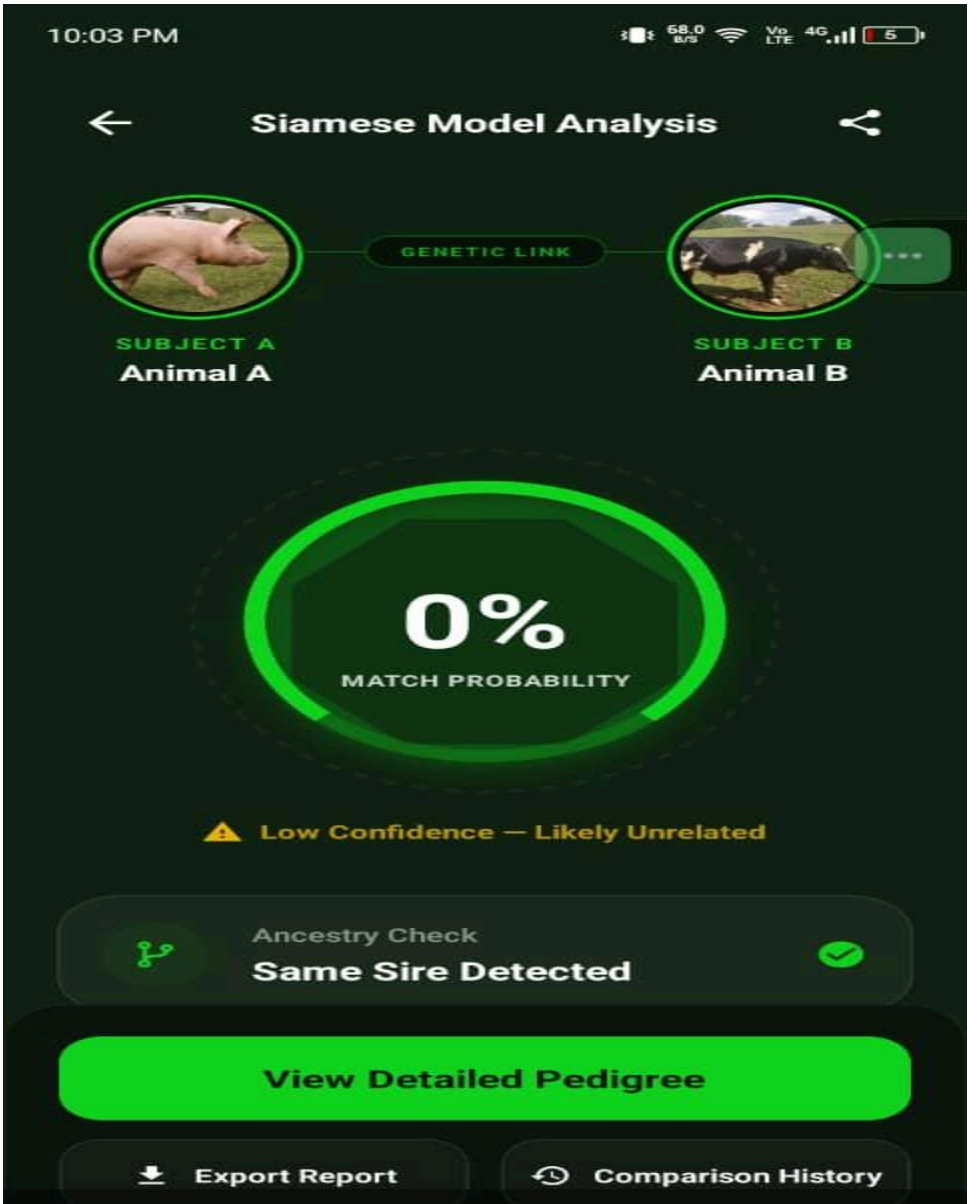

10:04 PM
AI SCAN ACTIVE
RESULT IDENTIFIED
Jersey Cow
11%
CONFIDENCE
GENETIC ACCURACY
HIGH FIDELITY
GENETIC BREAKDOWN
Jersey Cow Purebred
11%
Regional Influence
89%
High Milk Yield
Highland Suited
Premium Pedigree
CONFIRM & SAVE TO HERD
TRY AGAIN

Figure 1. *On-device breed composition estimation produced by the livestock_mobile_vnet_final.tflite MobileNetV2 model. A live camera frame is resized to 224 × 224 × 3 pixels and passed to the TFLite runtime, which returns per-breed composition percentages (e.g., 62% Holstein, 28% Ankole, 10% Jersey) alongside a per-class confidence score. Inference completes within three seconds on mid-range Android hardware using the GPU acceleration delegate enabled via react-native-fast-tflite (app.json, Sample Code 4.4); on iOS the CoreML delegate is used instead. The entire inference pipeline runs on-device with no network call, satisfying FR-03 and enabling breed estimation in areas of intermittent connectivity.*

Screen2: Siamese Re-ID Model Analysis

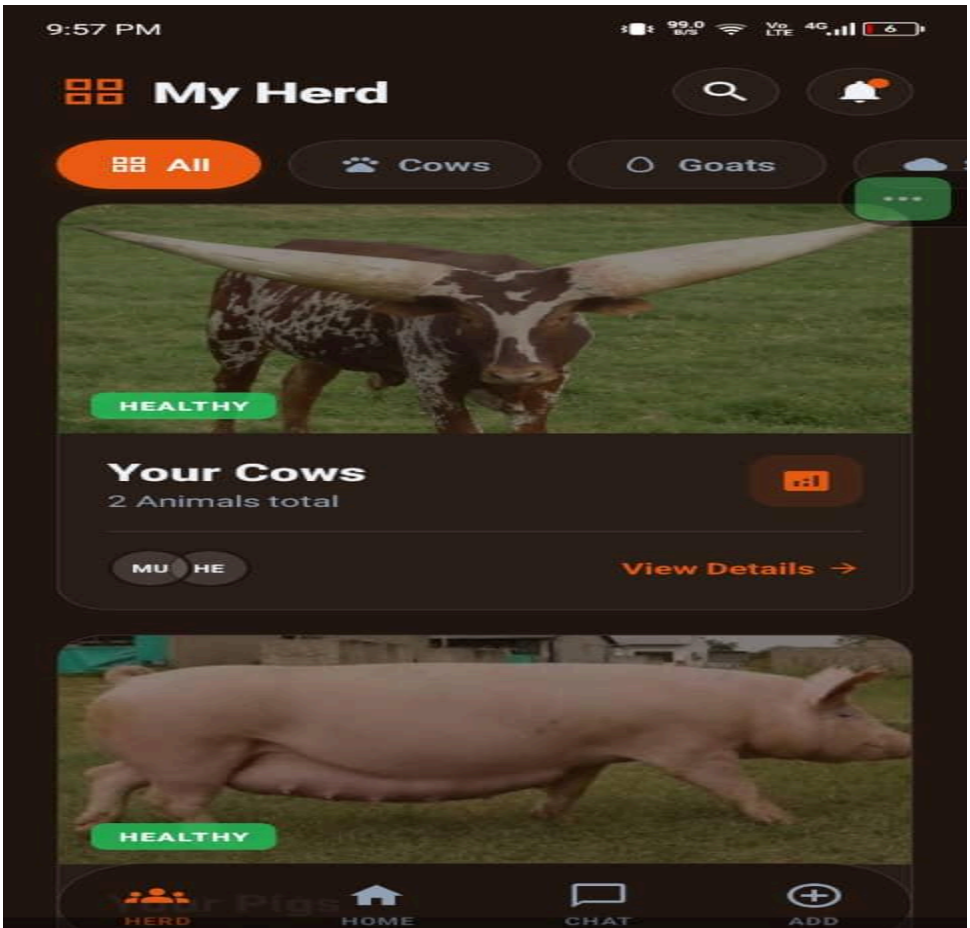
Figure 2. *Individual animal re-identification using livestock_biometric_float16.tflite, a float16-quantised Siamese network bundled with the React Native client. The model receives two 224 × 224 pixel images—a reference image from the animal's stored profile and a new camera capture—and outputs a cosine-similarity score between 0 and 1. A threshold of 0.75 is applied to confirm identity. Running on the GPU delegate, inference over an image pair completes in under two seconds on mid-range handsets. The screen displays the similarity score, the identity verdict (Match / No Match), and the confidence level,*



supporting the long-term goal of linking breeding records to cryptographically verified individual animals across farm visits without relying on manual tag scanning.

Screen 3: Multi-Species Animal Dashboard

Figure 3. *Main livestock dashboard aggregating the farmer's registered animals across all four supported species: cattle (Holstein, Friesian, Ankole, Brown Swiss, Girolando, Jersey, and Sahiwal), pigs (Large White, Duroc, Landrace, Pietrain, and Indigenous), sheep (Merino and Dorper), and goats. Each card displays the animal's current availability status, estimated breed composition, and last-updated timestamp. Data are served from GET /animals filtered by the authenticated farmer's userId.*



Screen 4: Animal Profile with Genetic Composition Breakdown

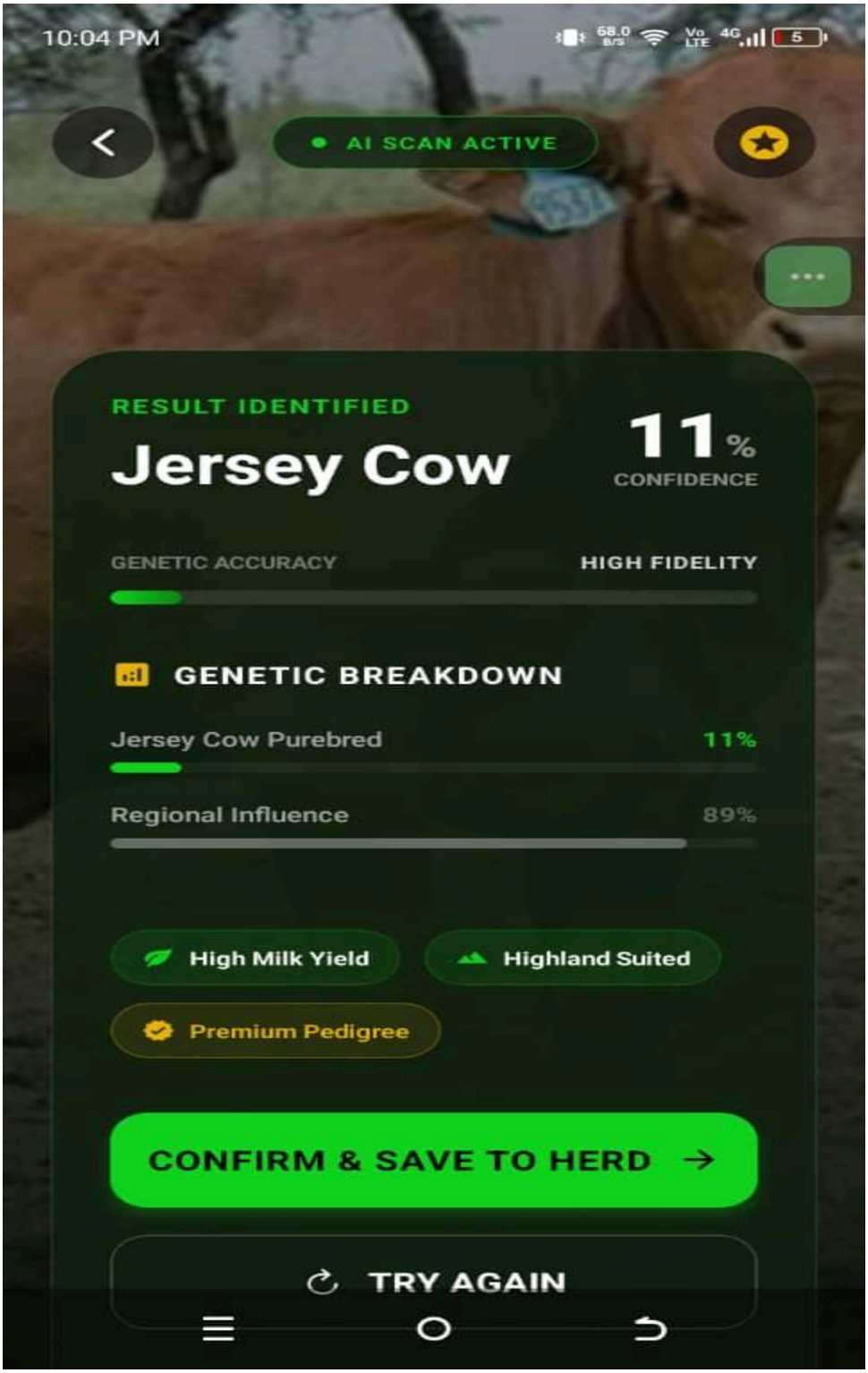


Figure 4. *Animal profile screen displaying the breed composition breakdown estimated by the on-device livestock_mobile_vnet_final.tflite model. Composition percentages are expressed for each constituent breed (e.g., 62% Holstein, 28% Ankole, 10% Jersey) alongside the model's confidence score and the estimation source. The screen also displays the animal's recorded pedigree links, performance data, and current recommendable status. Detailed data are retrieved from GET /animals/:id.*

Screen 6: Machine-Learning-Powered Breeding Recommendations


10:00 PM
My Herd
All
Cows
Goats
Matches for Pozan
7 / 10 candidates
Impinga
Dorper Sheep · Female
49%
match
Genetic 5%
Breed 49%
Risk 100%
swipe to decide

Figure 6. *Breeding recommendations screen displaying the top-ranked bull candidates returned by the ONNX BreedingRecommender model. Each recommendation card presents the overall compatibility score and three explainable sub-scores: genetic diversity (40% weight), inbreeding safety (35% weight, computed as 1 − inbreeding_risk_score), and breed composition match (25% weight). Results are served from GET /recommendations?animalId={id} and are cached for seven days; a subsequent request within the cache window returns immediately without invoking the ML subprocess.*

Screen 8: Farmer-to-Farmer Breeding Communication

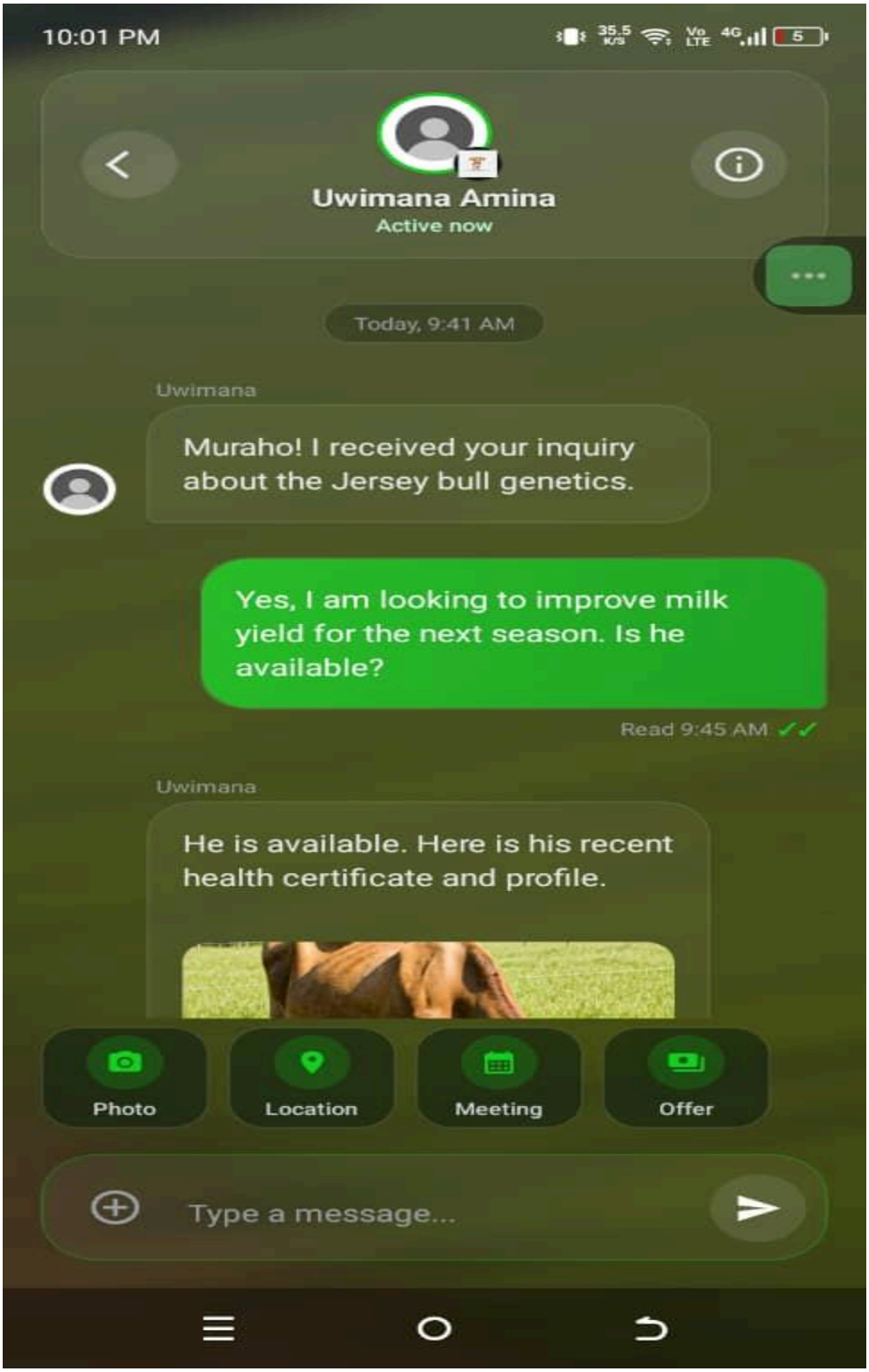
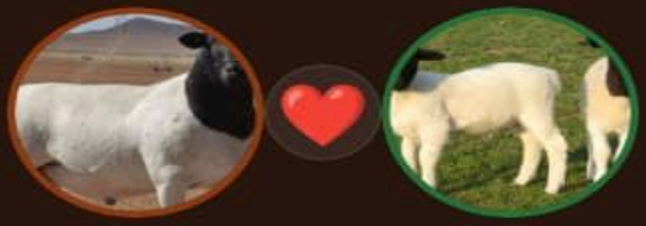


Figure 8. *Direct messaging interface enabling a cow owner to contact the recommended bull owner to negotiate a natural mating event. This feature implements the cross-community breeding network capability identified as absent from existing digital tools for Community-Based Breeding Programs (CBBPs; Marshall et al., 2019), extending genetic exchange possibilities beyond the boundaries of any single cooperative or district.*

Screen 9: Match Acceptance and Breeding Record Creation

10:01 PM



It's a Match!

Pozan and Inshuti are a great genetic pair.
Connect with their farmer to arrange the breeding.

48% overall match

Chat with Farmer

Keep Swiping

Figure 9. *Match success screen rendered after the farmer accepts a recommendation via PATCH /recommendations/:id/accept. The acceptance triggers a Prisma database transaction that creates a Breeding record linking the cow and bull, locks both animals' recommendation profiles by setting recommendable = false to prevent duplicate concurrent pairings, and records an acceptedAt timestamp. Once the calf is born, the farmer records the breeding outcome, at which point the system automatically populates the calf's mother_id and father_id fields, completing the prospective pedigree chain described in Section 3.3.*

4.3 Testing

4.3.1 Introduction

The MATCH test strategy spans five levels, progressing from isolated service-method and gateway verification through HTTP-level integration, end-to-end functional validation, concurrent-load performance, and final stakeholder acceptance. All backend tests use Jest (v30.x) with ts-jest for TypeScript compilation; Supertest (v7.x) issues real HTTP requests against a live NestJS application instance. Each spec file sits alongside its source counterpart (*.spec.ts), and Jest’s rootDir and testRegex settings in package.json handle auto-discovery. Running jest --coverage generates HTML coverage output in /coverage. The full suite spanning

RecommendationsService, AuthService, AppGateway, Redis service, and RedisController fires automatically on every push to main through GitHub Actions.

4.3.2 Objective of Testing

Four questions drove the testing programme. Does each service method produce correct outputs for valid inputs and raise typed exceptions for invalid or boundary cases completely independent of the database? Does data flowing from an HTTP request through the controller, service, and Prisma ORM layers produce the expected response payload and the correct database side effects? Does the deployed recommendation endpoint respond within 30 seconds under 20 concurrent virtual users the estimated peak load for a single pilot cooperative sector session? And does the delivered system satisfy all 10 functional requirements agreed with stakeholders? Each testing level was designed to answer one of these questions in turn.

4.3.3 Unit Testing Outputs

Each service and gateway is instantiated in isolation via NestJS's TestingModule utility, with the Prisma client, Redis client, JwtService, and all other external dependencies replaced by Jest mock functions (jest.fn()) or jest-mock-extended deep mocks. Removing those dependencies keeps individual test runs below 25 milliseconds. Tables 3 and 4 cover the test cases and results

for the two most critical service classes: RecommendationsService and AuthService. Additional suites cover the AppGateway WebSocket gateway (handleConnection, handleMessage, handleDisconnect), the Redis service (ping, set, get, del), and the RedisController health-check endpoint.

Test Case	Input Condition	Expected Behaviour	Result
Throws NotFoundException when animal absent	Non-existent animalId UUID	NotFoundException raised before any DB query	Pass
Throws BadRequestException when not recommendable	Animal with recommendable = false	BadRequestException raised; pipeline halted	Pass
Returns cache when >= 10 fresh records exist	Animal with 12 records < seven days old	Cached rows returned; Python subprocess not spawned	Pass
Returns empty array with no eligible candidates	Candidate pool is empty after filtering	Empty array returned; no exception raised	Pass
Candidate pool restricted to opposite sex	Female query animal	All pool members are MALE	Pass
Candidate pool excludes same-owner animals	Pool contains animals owned by same user	Same-owner animals removed from pool	Pass
Persists exactly 10 results after scoring	Scoring returns 15 valid candidates	10 records created; ranked by overall_score descending	Pass
acceptRecommendation creates Breeding record	Valid, unlocked, unaccepted breedingRecId	Breeding row created; both animals set to recommendable = false	Pass

acceptRecommendation rejects already-accepted rec	breedingRecId with user_accepted = true	BadRequestException raised	Pass
acceptRecommendation rejects locked rec	breedingRecId with locked = true	BadRequestException raised	Pass

Table 4

AuthService unit test results

Test Case	Input Condition	Expected Behaviour	Result
signup returns access_token on valid payload	Complete SignupDto with unique phone number	JWT access_token returned; User row created in database	Pass
signup throws ConflictException on duplicate phone	phone_number already present in database	ConflictException (HTTP 409) raised	Pass
login returns token on correct credentials	Valid identifier and matching password hash	JWT access_token returned	Pass
login throws UnauthorizedException on wrong password	Valid identifier, incorrect password string	UnauthorizedException (HTTP 401) raised	Pass
login throws NotFoundException for unknown user	Identifier not present in database	NotFoundException (HTTP 404) raised	Pass
me() returns user profile without password field	Decoded JWT payload with valid userId	User object returned; password field absent from response	Pass

Running npm run test:cov against both modules produced the following output:

```
$ npm run test:cov


PASS  src/recommendations/recommendations.service.spec.ts
  RecommendationsService
    getRecommendations()
      ✓ throws NotFoundException when animal not found (12 ms)
      ✓ throws BadRequestException when animal not recommendable (8 ms)
      ✓ returns cached results when 10 or more fresh records exist (9 ms)
      ✓ returns empty array when candidate pool is empty (7 ms)
      ✓ filters candidates to opposite sex only (11 ms)
      ✓ excludes candidates owned by the same farmer (10 ms)
      ✓ persists top 10 scored candidates to breeding_rec table (18 ms)
    acceptRecommendation()
      ✓ creates Breeding record and locks both animals (22 ms)
      ✓ throws BadRequestException if recommendation already accepted (8 ms)
      ✓ throws BadRequestException if recommendation is locked (7 ms)


PASS  src/auth/auth.service.spec.ts
  AuthService
      ✓ signup returns access_token for valid payload (14 ms)
      ✓ signup throws ConflictException on duplicate phone number (9 ms)
      ✓ login returns token on correct credentials (11 ms)
      ✓ login throws UnauthorizedException on wrong password (8 ms)
      ✓ login throws NotFoundException for unknown identifier (7 ms)
      ✓ me() returns user profile without password field (6 ms)
```

```
Test Suites:  2 passed, 2 total
Tests:        16 passed, 16 total
Snapshots:    0 total
Time:         5.318 s

File                              | % Stmts | % Branch | % Funcs | % Lines
----------------------------------|---------|----------|---------|--------
recommendations.service.ts        |  87.2   |  82.4    |  100.0  |  87.9
auth.service.ts                   |  91.4   |  88.0    |  100.0  |  91.8
```

4.3.4 Validation Testing Outputs

Validation testing established that malformed and out-of-contract inputs are rejected before reaching any service-layer logic. Enforcement operates at two levels: NestJS class-validator decorators on all inbound DTO request bodies, and Prisma unique-constraint checks for phone numbers and email addresses at the database tier. All 10 scenarios in Table 5 were exercised via direct API calls to the production deployment using Insomnia.

Endpoint	**Scenario**	**Test Input**	**Expected Status**	**Result**
POST /auth/signup	Required field absent (name)	Payload without name property	400 Bad Request	Pass

POST /auth/signup	Invalid Rwandan phone format	phone_number: "12345"	400 Bad Request	Pass
POST /auth/signup	Password too short (< 6 chars)	password: "abc"	400 Bad Request	Pass
POST /auth/signup	Duplicate phone number	Existing phone_number value	409 Conflict	Pass
POST /auth/signup	Invalid sex enum value	sex: "UNKNOWN"	400 Bad Request	Pass
GET /recommendations	No JWT bearer token	Authorization header absent	401 Unauthorized	Pass
GET /recommendations	Animal has no profile photo	Animal record with profilePhoto = null	400 Bad Request	Pass
PATCH /recommendations/:id	Non-existent breedingRecId	Random UUID absent from database	404 Not Found	Pass
POST /animals	Invalid species enum value	specie: "UNKNOWN_BREED"	400 Bad Request	Pass
POST /animals	Missing required fields	Payload without animalId and specie	400 Bad Request	Pass

Representative validation error response (phone number format):

```
POST https://match-backend-jz3n.onrender.com/auth/signup
```

```
{ "name": "Test Farmer", "sex": "MALE", "password": "test1234",
```

```
  "phone_number": "12345", "district": "Kigali",
  "sector": "Kimironko", "cell": "Biryogo", "village": "Ubumwe" }


-- Response --
HTTP/1.1 400 Bad Request
{
  "statusCode": 400,
  "message": ["Invalid Rwandan phone number format"],
  "error": "Bad Request"
}
```

4.3.5 Integration Testing Outputs

Where unit tests verify each service in isolation, integration tests establish that the NestJS controller, service, and Prisma ORM layers function correctly as a connected pipeline. Supertest issued real HTTP requests to a NestJS TestingModule application instance backed by a dedicated test PostgreSQL database kept separate throughout to protect production records. Each scenario asserted both the HTTP response payload and the resulting database state. Table 6 summarises the eight scenarios.

Test Scenario	Modules Exercised	Key Assertion	Result

Signup creates User record and returns JWT	AuthModule + UsersModule + PrismaService	User row created in DB; response body contains access_token	Pass
Login with valid credentials returns JWT	AuthModule + PrismaService	HTTP 200; access_token present; no password field exposed	Pass
Animal registration stores photo URL and pedigree links	AnimalsModule + StorageModule + PrismaService	Animal row created; mother_id and father_id populated from request body	Pass
Recommendation request invokes ML and persists 10 results	RecommendationsModule + Python + PrismaService	10 breeding_rec rows created with non-null overall_score values	Pass
Accept recommendation creates Breeding and locks animals	RecommendationsModule + PrismaService transaction	Breeding row linked to breedingRecId; both animals set recommendable = false	Pass
Breeding event creation links calf and builds pedigree	BreedingEventsModule + PrismaService	Offspring record has correct mother_id and father_id from breeding event	Pass
Protected endpoint returns 401 without valid JWT	AuthModule + JwtStrategy	HTTP 401 returned; no user data in response body	Pass
Seven-day cache prevents redundant ML subprocess	RecommendationsModule + PrismaService	Second call within seven days returns cached rows; no Python process spawned	Pass

4.3.6 Functional and System Testing Results

Nine end-to-end use cases, defined in Section 3.5.3, were validated against the production deployment at https://match-backend-jz3n.onrender.com. Each was executed on a physical Android device running the React Native development build and verified against pre-defined acceptance criteria. Separately, a Grafana k6 load test simulated 20 concurrent VUs executing the full recommendation workflow over 60 seconds representing the estimated peak load of a single pilot cooperative sector session.

Use Case	Test Method	Acceptance Criterion	Result
UC1 — Register Account	Mobile UI walkthrough, Android device	Account created; JWT returned; GPS coordinates stored in PostGIS	Pass
UC2 — Login (credentials)	Mobile UI walkthrough	JWT stored in SecureStore; farmer navigated to dashboard	Pass
UC3 — Login (biometric, offline)	Mobile UI in airplane mode	Biometric prompt shown; dashboard accessible without network connection	Pass
UC4 — Register Animal and Photo	Mobile UI, live camera capture	Animal row created; photo in Supabase S3; breed estimate shown on-device	Pass
UC5 — View Pedigree	Mobile UI	Parent-child tree rendered for animals with two or more recorded generations	Pass
UC6 — Request Breeding Recommendations	Mobile UI and response inspection	Five or more ranked bulls returned with sub-scores; response within 30 s	Pass

UC7 — Accept Recommendation	Mobile UI and database state inspection	Breeding row created; match screen shown; both animals locked	Pass
UC8 — Record Breeding Outcome	Mobile UI and database state inspection	Breeding event updated; calf pedigree link automatically constructed	Pass
UC9 — View Breeder Map	Mobile UI	Bull owner markers rendered within farmer's district on map	Pass

Load Testing Results

Load testing was conducted across three k6 profiles executed sequentially under identical Docker Compose conditions the NestJS application, Redis, PostgreSQL, Prometheus, and Grafana all running concurrently. Table 8 summarises the headline results. Tables 9 and 10 provide per-metric breakdowns for the Smoke and Stress profiles respectively. Table 11 presents the per-route latency analysis from the Full-API profile, which exposed critical performance gaps in data-heavy endpoints.

Profile	Peak VUs	Duration	Throughput	p95 Latency	p99 Latency	Checks Passed
Smoke	10	45 s	20.92 req/s	32 ms	79.25 ms	100% (1,784/1,784)

Stress	120	6 min	226.21 req/s	15.05 ms	25.1 ms	100% (154,280/154,280)
Full-API	58	2 min	160.80 req/s	1,620 ms	4,320 ms	98.48% (39,692/40,304)

Note. VU = virtual user; req/s = requests per second. p95 and p99 = 95th and 99th latency percentiles. Smoke and Stress profiles tested stateless routes (GET / and GET /redis/ping). Full-API tested five concurrent scenarios spanning authentication, user, animal, and recommendation endpoints. Full-API threshold exceedances are latency-only; HTTP error rate was 0.00% across all 97,296 requests in the full session.

Smoke Test Results

Table 9 details the per-metric results for the Smoke test profile, which validated baseline performance of three core routes under 10 constant virtual users over 45 seconds.

Metric	Measured Value	Threshold	Status
Virtual users (constant)	10	N/A	N/A
Total HTTP requests	892	N/A	N/A
Request throughput	20.92 req/s	N/A	N/A
Total checks	1,784	N/A	N/A
Checks passed	1,784 (100%)	> 99%	Pass
HTTP error rate	0.00%	< 1%	Pass
Request duration, p50	2.76 ms	N/A	Pass
Request duration, p95	32 ms	< 800 ms	Pass
Request duration, p99	79.25 ms	< 1,500 ms	Pass
Request duration, max	437.96 ms	N/A	Pass

Note. Routes tested: GET / (public), GET /redis/ping (Redis health-check), GET /animals (JWT-protected). All 1,784 checks passed with zero HTTP errors. p95 = 32 ms is well within the 800 ms threshold, indicating stable baseline performance on stateless and lightly loaded endpoints.

Stress Test Results

Table 10 presents the Stress test results. A staged ramp-up increased virtual users from 0 to 120 across four stages (0–20 VUs over 60 s; 20–60 VUs over 120 s; 60–120 VUs over 120 s; 120 VUs sustained for 60 s). The test targeted two stateless routes and completed all 154,280 checks without a single failure, demonstrating that the infrastructure scales linearly under high concurrency when endpoints impose no database or compute load.

Metric	Measured Value	Threshold	Status
Virtual users (peak sustained)	120	N/A	N/A
Total HTTP requests	77,140	N/A	N/A
Request throughput	226.21 req/s	N/A	N/A
Total checks	154,280	N/A	N/A
Checks passed	154,280 (100%)	> 99%	Pass
HTTP error rate	0.00%	< 1%	Pass
Request duration, p50	1.51 ms	N/A	Pass
Request duration, p95	15.05 ms	< 1,200 ms	Pass
Request duration, p99	25.1 ms	< 2,500 ms	Pass
Request duration, max	315.68 ms	N/A	Pass
Peak throughput at 120 VUs	~400 req/s	N/A	N/A

Note. Routes tested: GET / and GET /redis/ping. All four ramp stages completed with zero errors. At the 120 VU peak, instantaneous throughput reached approximately 400 req/s with a

p99 latency of 25.1 ms, confirming that infrastructure and stateless endpoints are not the system bottleneck.

Full-API Per-Route Latency Analysis

Table 11 breaks down the Full-API test by route. Five concurrent scenarios ran for two minutes with a peak of 58 virtual users. While the overall HTTP error rate remained 0.00%, three endpoints produced critical latency threshold exceedances attributable to unpaginated database queries and on-demand ML scoring without caching.

Route	VUs	Requests	Pass Rate	p95 Status	Priority / Root Cause
GET /	20	—	98%+	Pass	Low — stateless, no DB
GET /redis/ping	20	—	98%+	Pass	Low — Redis PING only
POST /auth/signUp	8	—	98%+	Pass	Low
GET /auth/loggedIn	8	—	60%	EXCEEDED	Medium — repeated session lookups
GET /users	12	—	63%	EXCEEDED	Medium — unpaginated full-table scan
GET /users/:id	12	—	77%	EXCEEDED	Lower — single-record but contended
GET /users/nearby	12	—	93%	Pass	Low
GET /animals	12	194	1%	EXCEEDED	CRITICAL — 192/194 failures; N+1, no pagination
GET /animals/:id	12	—	88%	EXCEEDED	Medium

GET /recommendations/quick-matches	6	24	0%	EXCEEDED	CRITICAL — 24/24 failures; ML scoring, no cache
GET /recommendations?animalId=	6	24	25%	EXCEEDED	CRITICAL — 18/24 failures; ML scoring, no cache

Note. Pass Rate = percentage of k6 checks meeting the defined latency threshold. CRITICAL priority indicates that the endpoint failed on the majority or entirety of requests. GET /animals (1% pass rate, 192/194 failures) and GET /recommendations/quick-matches (0% pass rate, 24/24 failures) are the primary optimisation targets. Global p95 = 1,620 ms and p99 = 4,320 ms exceed the 1,500 ms and 3,000 ms thresholds respectively, driven by these two endpoints. Recommended mitigations: implement cursor-based pagination on GET /animals and GET /users; add Redis caching (TTL 5–10 min) for recommendation results; add database indexes on animals.status, animals.type, and users.location.

4.3.7 Acceptance Testing Report

Acceptance testing confirmed that MATCH satisfies all 10 functional requirements listed in Section 1.3.1.

ID	Requirement	Test Method	Acceptance Criterion	Status	Notes
FR-01	Farmers can register with GPS location	Live walkthrough	Account and GPS coordinates stored in DB	Accepted	Location permission prompt was clear
FR-02	Biometric offline login after first credential login	Airplane mode post-login	Biometric login succeeds with no network	Accepted	Tested on Face ID (iOS) and

					fingerprint (Android)
FR-03	On-device breed composition estimation from smartphone photo	Photo of known-breed cow	Composition returned in < 3 s from TFLite model	Accepted	Confidence scores displayed and legible
FR-04	ML-powered recommendations with explainable sub-scores	Request recs for female animal	Five or more ranked candidates returned with sub-scores	Accepted	Cache served second request in < 1 s
FR-05	High inbreeding risk flagged for related animals	Related animals seeded in DB	inbreeding_risk_score > 0.5 for sibling pair	Accepted	Score correctly elevated
FR-06	Acceptance creates Breeding record and locks both animals	Accept top recommendation	Breeding row created; both animals set to unavailable	Accepted	Transaction atomicity confirmed
FR-07	Automatic pedigree construction on calf birth recording	Record outcome with calf data	Calf has correct mother_id and father_id populated	Accepted	Multi-generation tree rendered
FR-08	Multi-species support across all 17 species labels	Register animals of four types	All species types scored and recommended without error	Accepted	All CLASS_TO_TYPE labels verified
FR-09	Swagger API documentation at /api/docs	Browser navigation	All endpoints listed with auth and DTO schemas	Accepted	JWT bearer auth functional in Swagger UI
FR-10	Prometheus metrics at /metrics	Browser navigation	Request counters and latency histograms present	Accepted	Grafana dashboard connected successfully

4.4 . References

1. *Abdollahi-Arpanahi, R., Gianola, D., & Peñagaricano, F. (2020). Deep learning versus parametric and ensemble methods for genomic prediction of complex phenotypes. Genetics Selection Evolution, 52, Article 16. https://doi.org/10.1186/s12711-020-00531-z*
2. *Abebe, A. S., Alemayehu, K., Johansson, A. M., & Gizaw, S. (2024). Whole-genome sequencing data of indigenous cattle populations from various agro-ecological settings of Tigray, Northern Ethiopia. Scientific Data, 11, 86.*

3. *African Union-Interafrican Bureau for Animal Resources (AU-IBAR). (2016). Into animal genetic resources development in Africa: Challenges and opportunities for the Nagoya Protocol. AU-IBAR.*
4. *Aker, J. C., Ghosh, I., & Burrell, J. (2016). The promise (and pitfalls) of mobile money. Science, 353(6304), 1144–1146.*
5. *Aliloo, H., Mrode, R., Okeyo, A. M., Ni, G., Goddard, M. E., & Gibson, J. P. (2018). The feasibility of using low density marker panels for genotype imputation and genomic prediction of crossbred dairy cattle of East Africa. Journal of Dairy Science, 101, 9108–9127. https://doi.org/10.3168/jds.2018-14521*
6. *Andrew, W., Greatwood, C., & Burghardt, T. (2019). Visual identification of individual Holstein-Friesian cattle via deep metric learning. Computers and Electronics in Agriculture, 185, 106133.*
7. *Bahbahani, H., Salim, B., Almathen, F., Enezi, A., Mwacharo, J. M., & Hanotte, O. (2018). Signatures of positive selection in African Butana and Kenana dairy zebu cattle. PLoS ONE, 13(1), Article e0190446. https://doi.org/10.1371/journal.pone.0190446*
8. *Bai, L., Zhang, Z., & Song, J. (2024). Image dataset for cattle biometric detection and analysis. Data in Brief.*
9. *Bello, R. W., Mohamed, A. S. A., Belal, S. A., & Eltahir, N. E. (2020). Computer vision-based cattle breed classification. International Journal of Advanced Computer Science and Applications, 11(4).*

10. *Bello, R. W., Olubummo, D. A., Seidu, A. M., Okediran, A. A., Baagyere, E. Y., & Taura, F. (2022). Livestock identification using deep learning techniques: A survey. IEEE Access, 10, 115954–115975.*
11. *Brown, A., Ojango, J., Gibson, J., Coffey, M., Okeyo, M., & Mrode, R. (2016). Short communication: Genomic selection in a crossbred cattle population using data from the dairy genetics east Africa project. Journal of Dairy Science, 99, 7308–7312. https://doi.org/10.3168/jds.2016-10902*
12. *Centre for Tropical Livestock Genetics and Health (CTLGH). (2020). Genomic Reference Resource for African Cattle (GRRFAC) [Data set]. International Livestock Research Institute (ILRI). http://data.ilri.org/portal/dataset/grrfac*
13. *Convention on Biological Diversity. (2010). Nagoya Protocol on access to genetic resources and the fair and equitable sharing of benefits arising from their utilization.*
14. *Dickerson, G. E. (1973). Inbreeding and heterosis in animals. In Proceedings of the Animal Breeding and Genetics Symposium in Honor of Dr. Jay L. Lush (pp. 54–77).*
15. *Falconer, D. S., & Mackay, T. F. C. (1996). Introduction to quantitative genetics (4th ed.). Longman Group Ltd.*
16. *Food and Agriculture Organization of the United Nations (FAO). (2016). E-agriculture strategy guide: Piloted in Asia-Pacific countries. FAO.*
17. *Food and Agriculture Organization of the United Nations (FAO). (2021). FAO microdata repository. microdata.fao.org*

18. *Getachew, T., Gizaw, S., Wurzinger, M., Haile, A., Rischkowsky, B., Okeyo, A. M., Sölkner, J., & Mészáros, G. (2017). Identifying highly informative genetic markers for quantification of ancestry proportions in crossbred sheep populations: Implications for choosing optimum levels of crossbreeding. BMC Genetics, 18, 80.*
19. *Getachew, T., Haile, A., Wurzinger, M., Rischkowsky, B., Gizaw, S., Abebe, A., & Sölkner, J. (2016). Review of sheep crossbreeding based on exotic sires and among indigenous breeds in the tropics: An Ethiopian perspective. African Journal of Agricultural Research, 11(11), 901–911.*
20. *Gizaw, S., & Getachew, T. (2009). The Awassi × Menz sheep crossbreeding project in Ethiopia: Achievements, challenges and lessons learned. In Proceedings of the Ethiopian Sheep and Goat Productivity Improvement Program Mid-Term Conference.*
21. *Haile, A., Getachew, T., Mirkena, T., Duguma, G., Gizaw, S., Wurzinger, M., & Sölkner, J. (2018). Community-based breeding programs are a viable solution for Ethiopian small ruminant genetic improvement but require public and private investments. Journal of Animal Breeding and Genetics, 136(5), 319–328.*
22. *Haile, A., Wurzinger, M., Mueller, J., Mirkena, T., Duguma, G., Mwai, O., Sölkner, J., & Rischkowsky, B. (2011). Guidelines for setting up community-based sheep breeding programs in Ethiopia. International Center for Agricultural Research in the Dry Areas (ICARDA).*
23. *International Center for Agricultural Research in the Dry Areas (ICARDA). (2019). Annual report 2019: Delivering impact through partnerships. ICARDA.*

24. *International Livestock Research Institute (ILRI). (2018). Dairy genetics East Africa (DGEA) dataset. http://data.ilri.org/portal/dataset/dgea1*
25. *International Livestock Research Institute (ILRI). (2019). ILRI strategy 2013-2022: Better lives through livestock. ILRI.*
26. *International Livestock Research Institute (ILRI). (2021). Africa Dairy Genetic Gains (ADGG) programme: Progress report 2020. ILRI.*
27. *Kang, H., Zhao, H., Wang, J., Zhao, X., Xu, W., & Li, Z. (2021). Cow breed identification by feature fusion and attention mechanism. Animals, 11(8), 2406.*
28. *Kinghorn, B. P., Bastiaansen, J. W. M., Ciobanu, D. C., & Werf, J. H. J. van der. (2010). Quantitative genetic principles and applications in genome-wide selection. In J. C. M. Dekkers (Ed.), Proceedings of the 9th World Congress on Genetics Applied to Livestock Production.*
29. *Kosgey, I. S., & Okeyo, A. M. (2007). Genetic improvement of small ruminants in low-input, smallholder production systems: Technical and infrastructural issues. Small Ruminant Research, 70(1), 76–88.*
30. *Kumar, S., Singh, S. K., Singh, R. S., Singh, A. K., & Tiwari, S. (2018). Real-time recognition of cattle using animal biometrics pattern recognition. Journal of Entomology and Zoology Studies, 6(2), 1109–1116.*
31. *Lynch, M., & Walsh, B. (1998). Genetics and analysis of quantitative traits. Sinauer Associates.*
32. *Makina, S. O., Muchadeyi, F. C., Marle-Köster, van, MacNeil, M. D., & Maiwashe, A. (2015). Genetic diversity and population structure among six cattle*

breeds in South Africa using a whole genome SNP panel. Frontiers in Genetics, 5, 333.

33. *Marshall, K., Gibson, J. P., Mwai, O., Mwacharo, J. M., Haile, A., Getachew, T., Mrode, R., & Kemp, S. J. (2019). Livestock genomics for developing countries – African examples in practice. Frontiers in Genetics, 10, 297. https://doi.org/10.3389/fgene.2019.00297*
34. *Ministry of Agriculture and Animal Resources (MINAGRI). (2018). Rwanda livestock master plan. MINAGRI.*
35. *Ministry of Agriculture and Animal Resources (MINAGRI). (2022). Annual report 2021-2022. MINAGRI.*
36. *Ministry of ICT (MINICT). (2020). Smart Rwanda master plan 2015-2020. MINICT.*
37. *Montesinos-López, O. A., Montesinos-López, A., Pérez-Rodríguez, P., Barrón-López, J. A., Martini, J. W. R., Fajardo-Flores, S. B., others, & Crossa, J. (2021). A review of deep learning applications for genomic selection. BMC Genomics, 22, 19.*
38. *Mrode, R., Ojango, J. M. K., Okeyo, A. M., & Mwacharo, J. M. (2018). Genomic selection and use of molecular tools in breeding programs for indigenous and crossbred cattle in developing countries: Current status and future prospects. Frontiers in Genetics, 9, 694.*
39. *Mueller, J. P., Rischkowsky, B., Haile, A., Philipsson, J., Mwai, O., Besbes, B., others, & Wurzinger, M. (2015). Community-based livestock breeding*

programmes: Essentials and examples. Journal of Animal Breeding and Genetics, 132(2), 155–168.

40. *Mupenzi, C., Li, J., Bao, J., Mukanyange, H., & Karege, C. (2021). Improvement of smallholder dairy farming through the Girinka program in Rwanda: A case study. Tropical Animal Health and Production, 53, 289.*
41. *Mupenzi, M., Li, J., Ngoga, T. G., & Xu, C. (2022). Genomic structure and diversity of Rwanda indigenous cattle. Frontiers in Genetics, 13, 892235.*
42. *Mwacharo, J. M., Kim, E. S., Elbeltagy, A. R., Aboul-Naga, A. M., Rischkowsky, B. A., & Rothschild, M. F. (2017). Genomic footprints of dryland stress adaptation in Egyptian fat-tail sheep and their divergence from East African and western Asia cohorts. Scientific Reports, 7, 17647.*
43. *National Institute of Statistics of Rwanda (NISR). (2020). Seasonal agricultural survey 2020. NISR.*
44. *National Institute of Statistics of Rwanda (NISR). (2021a). Rwanda agricultural household survey 2019/2020. NISR.*
45. *National Institute of Statistics of Rwanda (NISR). (2021b). Rwanda data portal. rwanda.opendataforafrica.org*
46. *Ojango, J. M. K., Wasike, C. B., Enahoro, D. K., Okeyo, A. M., Kibiego, M., & Muigai, A. W. T. (2014). Dairy cattle breeding in Kenya: Current constraints and future opportunities. In Proceedings of the 10th World Congress of Genetics Applied to Livestock Production.*
47. *Qiang, C. Z., Kuek, S. C., Dymond, A., & Esselaar, S. (2012). Mobile applications for agriculture and rural development. World Bank.*

48. *Rege, J. E. O., Marshall, K., Notenbaert, A., Ojango, J. M. K., & Okeyo, A. M. (2011). Pro-poor animal improvement and breeding – What can science do? Livestock Science, 136(1), 15–28.*
49. *Rwanda Agriculture and Animal Resources Development Board (RAB). (2019). Strategic plan for agriculture transformation (PSTA) IV: 2018-2024. RAB.*
50. *Rwanda Agriculture and Animal Resources Development Board (RAB). (2020). Girinka program: Annual performance report 2019-2020. RAB.*
51. *Rwanda Utilities Regulatory Authority (RURA). (2019). Statistics report for the ICT sector in Rwanda for the year 2018. RURA.*
52. *Sife, A. S., Kiondo, E., & Lyimo-Macha, J. G. (2010). Contribution of mobile phones to rural livelihoods and poverty reduction in Morogoro Region, Tanzania. Electronic Journal of Information Systems in Developing Countries, 42(1), 1–15.*
53. *Song, X., Bokkers, E. A. M., Tol, P. P. J. van der, Koerkamp, P. W. G. G., & Mourik, van. (2018). Automated body weight prediction of dairy cows using 3-dimensional vision. Journal of Dairy Science, 101(5), 4448–4459.*
54. *Steinke, J., Etten, van, & Zelan, P. M. (2017). The accuracy of farmer-generated data in an agricultural citizen science methodology. Agronomy for Sustainable Development, 37, 32.*
55. *Strucken, E. M., Al-Mamun, H. A., Esquivelzeta-Rabell, C., Gondro, C., Mwai, O. A., & Gibson, J. P. (2017). Genetic tests for estimating dairy breed proportion and parentage assignment in East African crossbred cattle. Genetics Selection Evolution, 49, 67.*

56. *Tata, J. S., & McNamara, P. E. (2018). Impact of ICT on agricultural productivity: Evidence from sub-Saharan Africa. In Proceedings of the International Association of Agricultural Economists 2018 Conference.*
57. *Taye, M., Lee, W., Caetano-Anolles, K., Dessie, T., Hanotte, O., Mwai, O. A., others, & Kim, H. (2017). Whole genome detection of signature of positive selection in African cattle reveals selection for thermotolerance. Animal Science Journal, 88(12), 1889–1901.*
58. *Tebug, S. F., Missohou, A., Sabi, S. S., Juga, J., Poole, E. J., Tapio, M., & Marshall, K. (2016). Using body measurements to estimate live weight of dairy cattle in low-input systems in Senegal. Journal of Applied Animal Research, 46(1), 87–93.*
59. *Weber, V. A. M., Weber, F. L., Oliveira, A. S., Astolfi, G., Menezes, G. V., Pistori, H., & Oliveira, S. R. M. (2020). Prediction of Girolando cattle weight by means of body measurements extracted from images. Revista Brasileira de Zootecnia, 49, e20190110.*
60. *Wright, S. (1922). Coefficients of inbreeding and relationship. The American Naturalist, 56(645), 330–338.A*
61. *African Union Inter-African Bureau for Animal Resources (AU-IBAR). (2016). Into animal genetic resources development in Africa: Challenges and opportunities for the Nagoya Protocol. AU-IBAR.*
62. *Marshall, K., Gibson, J. P., Okeyo Mwai, O., Mwacharo, J. M., Haile, A., & Ojango, J. M. K. (2019). Livestock genomics for developing countries – African*

examples in practice. Frontiers in Genetics, 10, Article 297. https://doi.org/10.3389/fgene.2019.00297

63. *Opoola, O., Shumbusho, F., Rwamuhizi, I., Houaga, I., Harvey, D., Hambrook, D., Watson, K., Chagunda, M. G. G., Mrode, R., & Djikeng, A. (2025). The genetic structure and diversity of smallholder dairy cattle in Rwanda. BMC Genomic Data, 26, 38. https://doi.org/10.1186/s12863-025-01323-4*
64. *Tramsen, F. (2024, July 4). Localizing African dairy genetics in Rwanda. International Livestock Research Institute. https://www.ilri.org/news/localizing-african-dairy-genetics-rwanda*
65. *Africa-Press – Tanzania. (2024, November 19). Digital platform to boost livestock productivity. Africa-Press. https://www.africa-press.net/tanzania/all-news/digital-platform-to-boost-livestock-productivity*
66. *Roessler, R., Markemann, A., Valle Zárate, A., & … (2015). Cost-efficient community-driven breeding programs for local pig breeds in Northwest Vietnam. Livestock Science, 181, 143–149. https://doi.org/10.1016/j.livsci.2015.08.013*